Data Extraction

```
# Data Extraction.
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import pandas as pd
df = pd.read_csv('drive/My Drive/Tufts/Masters Thesis Research/Datasets/vehicles.csv')
df_copy = df.copy()
df.head()
```

	id	url	region	region_url	price	year	manufacturer	model	condition	cylinders
0	7222695916	https://prescott.craigslist.org/cto/d/prescott...	prescott	https://prescott.craigslist.org	6000	NaN	NaN	NaN	NaN	NaN
1	7218891961	https://fayar.craigslist.org/ctd/d/bentonville...	fayetteville	https://fayar.craigslist.org	11900	NaN	NaN	NaN	NaN	NaN
2	7221797935	https://keys.craigslist.org/cto/d/summerland-k...	florida keys	https://keys.craigslist.org	21000	NaN	NaN	NaN	NaN	NaN
3	7222270760	https://worcester.craigslist.org/cto/d/west-br...	worcester / central MA	https://worcester.craigslist.org	1500	NaN	NaN	NaN	NaN	NaN
4	7210384030	https://greensboro.craigslist.org/cto/d/trinit...	greensboro	https://greensboro.craigslist.org	4900	NaN	NaN	NaN	NaN	NaN

5 rows × 26 columns

```
# Dataset information.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 426880 entries, 0 to 426879
Data columns (total 26 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   id            426880 non-null  int64
 1   url           426880 non-null  object
 2   region        426880 non-null  object
 3   region_url    426880 non-null  object
 4   price         426880 non-null  int64
 5   year          425675 non-null  float64
 6   manufacturer  409234 non-null  object
 7   model         421603 non-null  object
 8   condition     252776 non-null  object
 9   cylinders     249202 non-null  object
```

```
 10  fuel          423867 non-null  object
 11  odometer      422480 non-null  float64
 12  title_status  418638 non-null  object
 13  transmission  424324 non-null  object
 14  VIN           265838 non-null  object
 15  drive         296313 non-null  object
 16  size          120519 non-null  object
 17  type          334022 non-null  object
 18  paint_color   296677 non-null  object
 19  image_url     426812 non-null  object
 20  description   426810 non-null  object
 21  county        0 non-null       float64
 22  state         426880 non-null  object
 23  lat           420331 non-null  float64
 24  long          420331 non-null  float64
 25  posting_date  426812 non-null  object
dtypes: float64(5), int64(2), object(19)
memory usage: 84.7+ MB
```

```
# New numeric columns
df['age'] = 2025-df['year']
df['price_per_mileage'] = df['price']/df['odometer']
df['mileage_per_year'] = df['odometer']/df['age']

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 426880 entries, 0 to 426879
Data columns (total 29 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   id                 426880 non-null  int64
 1   url                426880 non-null  object
 2   region             426880 non-null  object
 3   region_url         426880 non-null  object
 4   price              426880 non-null  int64
 5   year               425675 non-null  float64
 6   manufacturer       409234 non-null  object
 7   model              421603 non-null  object
 8   condition          252776 non-null  object
 9   cylinders          249202 non-null  object
 10  fuel               423867 non-null  object
 11  odometer           422480 non-null  float64
 12  title_status       418638 non-null  object
 13  transmission       424324 non-null  object
 14  VIN                265838 non-null  object
 15  drive              296313 non-null  object
 16  size               120519 non-null  object
 17  type               334022 non-null  object
 18  paint_color        296677 non-null  object
 19  image_url          426812 non-null  object
 20  description        426810 non-null  object
 21  county             0 non-null       float64
```

```
 22  state             426880 non-null  object
 23  lat               420331 non-null  float64
 24  long              420331 non-null  float64
 25  posting_date      426812 non-null  object
 26  age               425675 non-null  float64
 27  price_per_mileage  421629 non-null  float64
 28  mileage_per_year   421344 non-null  float64
dtypes: float64(8), int64(2), object(19)
memory usage: 94.4+ MB
```

```
# Get missing values across
df.isnull().sum()
```

	0
id	0
url	0

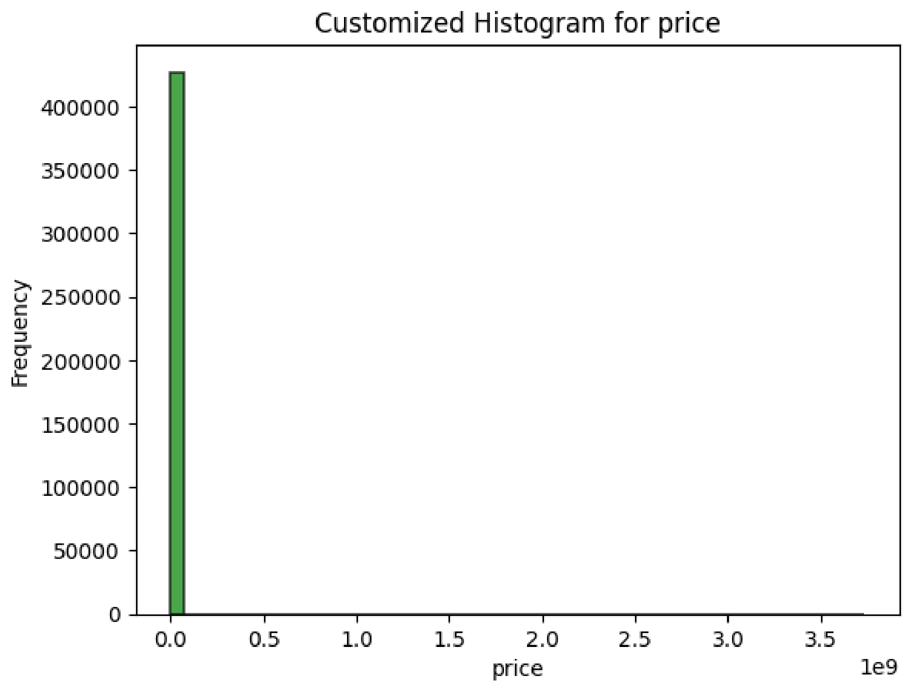
Plot Numeric Data, Impute from Median.

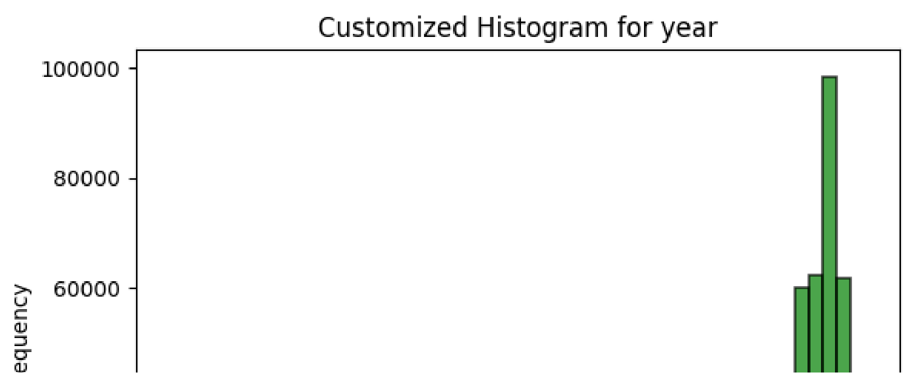
```
# Handling the missing data.
import matplotlib.pyplot as plt
import numpy as np
from sklearn.impute import SimpleImputer

df=df.drop('county', axis=1)
df=df.drop('id', axis=1)

numeric_columns = df.select_dtypes(include=np.number).columns.tolist()
print(numeric_columns)

# Impute The Missing Data - Moved before QuantileTransformer
for numeric_column in numeric_columns:
    if numeric_column == 'id':
        continue
    # Replace infinite values that might result from division by zero
    df[numeric_column].replace([np.inf, -np.inf], np.nan, inplace=True)
    df[numeric_column].fillna(df[numeric_column].median(), inplace=True)

# Create histogram for the numeric columns.
for column in numeric_columns:
  if column == 'id':
    continue

  # Create histogram to obtain normal distribution of the numeric variables.
  plt.figure()
  plt.hist(df[column], bins=50, color='green', alpha=0.7, edgecolor='black', linewidth=1.2)
  plt.title(f'Customized Histogram for {df[column].name}')
  plt.xlabel(f'{df[column].name}') # Removed (Transformed)
  plt.ylabel('Frequency')
  plt.show()
```

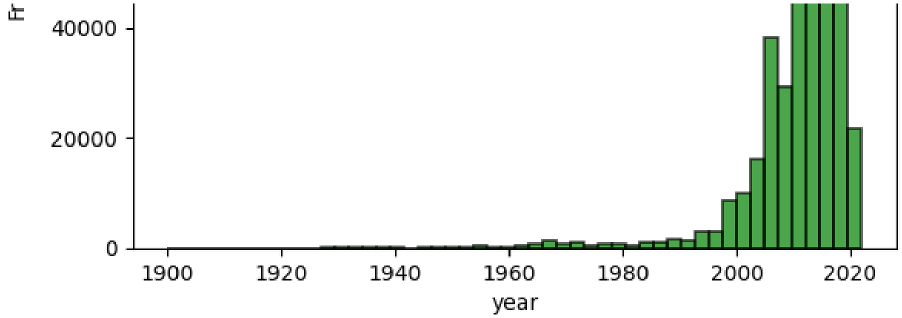
county	426880
state	0
lat	6549
long	6549
posting_date	68
age	1205
price_per_mileage	5251
mileage_per_year	5536

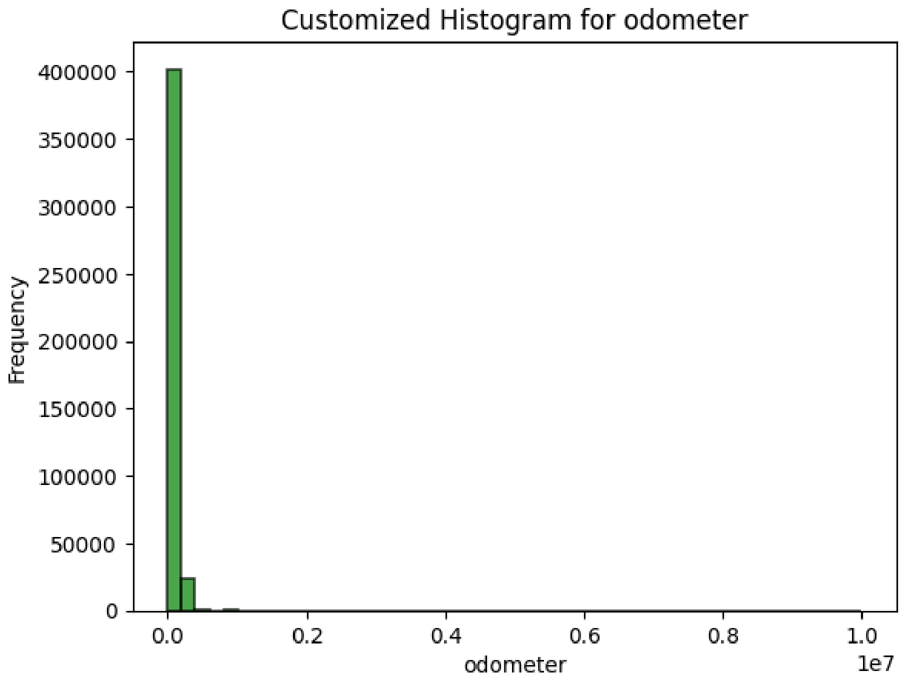
dtype: int64

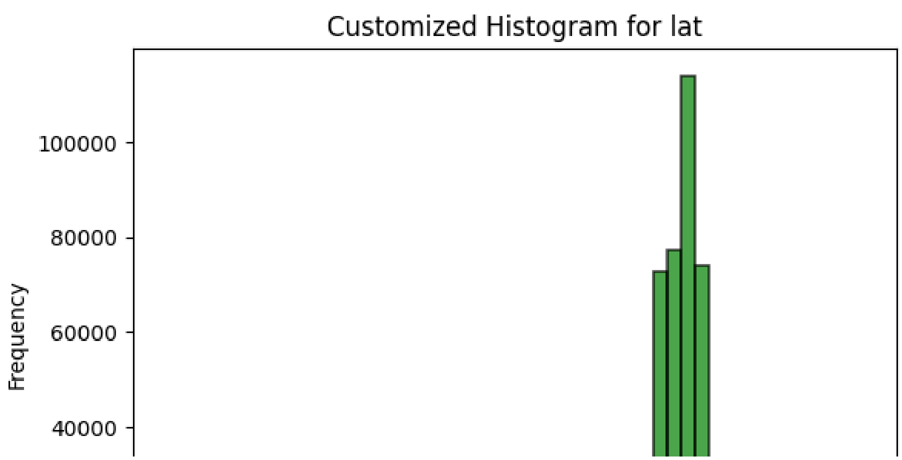
```
['price', 'year', 'odometer', 'lat', 'long', 'age', 'price_per_mileage', 'mileage_per_year']
/tmp/ipython-input-2091105120.py:17: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assig
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me


  df[numeric_column].replace([np.inf, -np.inf], np.nan, inplace=True)
/tmp/ipython-input-2091105120.py:18: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assig
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me


  df[numeric_column].fillna(df[numeric_column].median(), inplace=True)
```

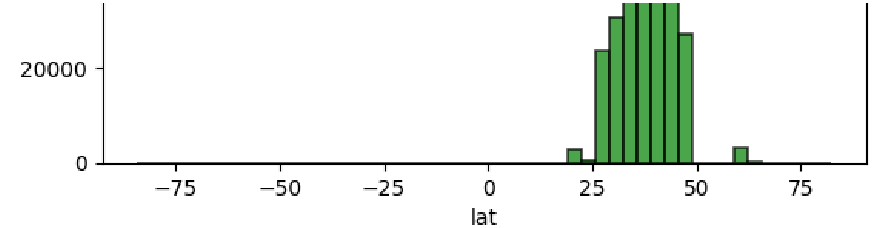


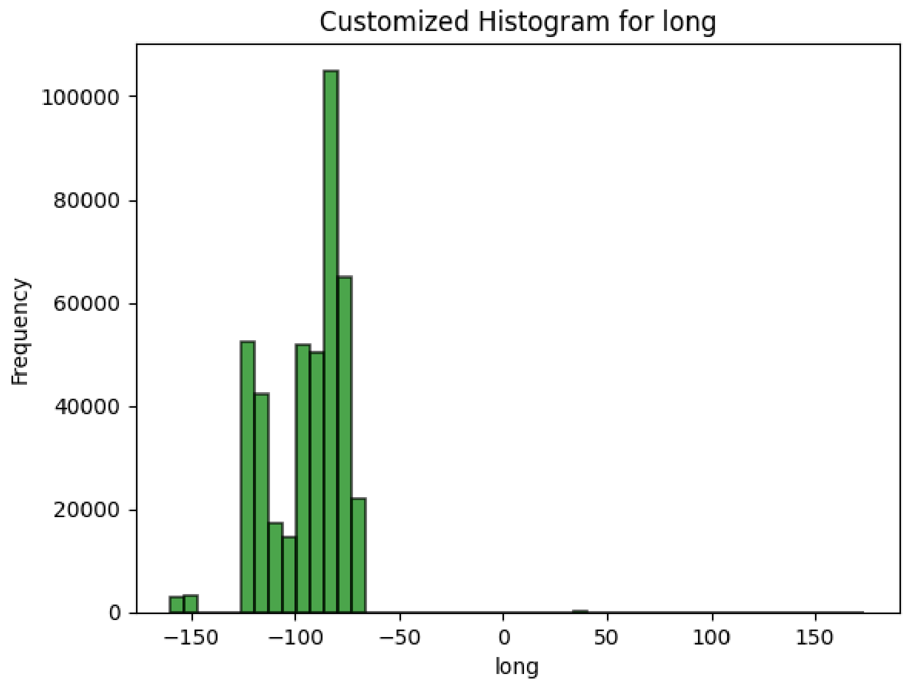


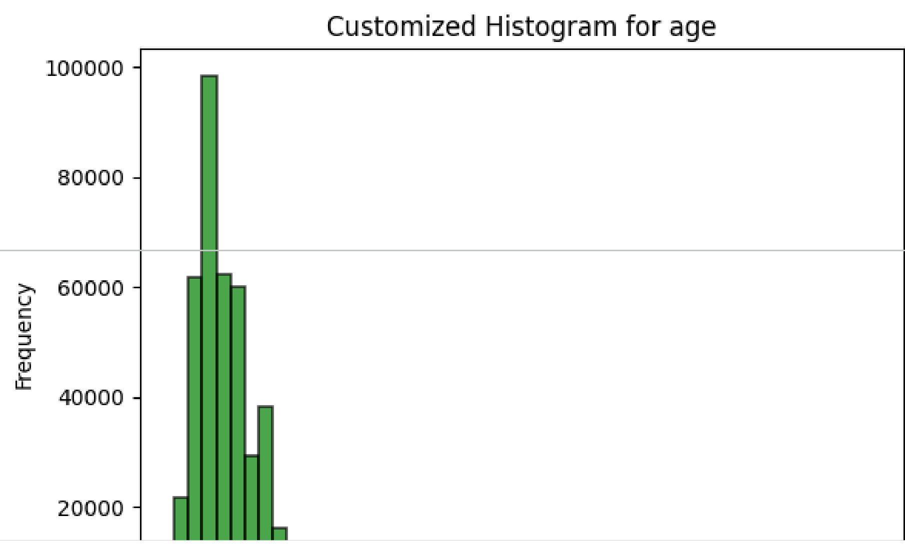

Fr
40000
20000
0
1900
1920
1940
1960
1980
2000
2020
year


Customized Histogram for odometer
Frequency
400000
350000
300000
250000
200000
150000
100000
50000
0
0.0
0.2
0.4
0.6
0.8
1.0
odometer
1e7


Customized Histogram for lat
Frequency
100000
80000
60000
40000


20000
0
−75
−50
−25
0
25
50
75
lat


Customized Histogram for long
Frequency
100000
80000
60000
40000
20000
0
−150
−100
−50
0
50
100
150
long


Customized Histogram for age
Frequency
100000
80000
60000
40000
20000

```
# Impute The Missing Data
for numeric_column in numeric_columns:
  if numeric_column == 'id':
    continue

  df[numeric_column].fillna(df[numeric_column].median(), inplace=True)

df.describe()
```

```
/tmp/ipython-input-4102822534.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignm
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me

  df[numeric_column].fillna(df[numeric_column].median(), inplace=True)
```

	price	year	odometer	lat	long	age	price_per_mileage	mileage_per_year
count	4.268800e+05	426880.000000	4.268800e+05	426880.000000	426880.000000	426880.000000	4.268800e+05	4.268800e+05
mean	7.519903e+04	2011.240173	9.791454e+04	38.504007	-94.651702	13.759827	9.763864e+01	7.312528e+03
std	1.218228e+07	9.439234	2.127801e+05	5.797112	18.240566	9.439234	9.211274e+03	1.612666e+04
min	0.000000e+00	1900.000000	0.000000e+00	-84.122245	-159.827728	3.000000	0.000000e+00	0.000000e+00
25%	5.900000e+03	2008.000000	3.813000e+04	34.757016	-111.907973	8.000000	4.405941e-02	4.017807e+03
50%	1.395000e+04	2013.000000	8.554800e+04	39.150100	-88.432600	12.000000	1.489615e-01	6.785056e+03
75%	2.648575e+04	2017.000000	1.330000e+05	42.350000	-81.030000	17.000000	5.824243e-01	9.404200e+03
max	3.736929e+09	2022.000000	1.000000e+07	82.390818	173.885502	125.000000	5.000000e+06	3.333333e+06

Imputing the Categorical Data

```
from sklearn.impute import SimpleImputer
categorical_columns = df.select_dtypes(include=['object']).columns
primary_columns = ['url', 'region_url', 'image_url', 'VIN']
categorical_columns = [column for column in categorical_columns if column not in primary_columns]

# Create histogram for the numeric columns.
for column in categorical_columns:
  if column == 'id':
    continue

  df[column] = SimpleImputer(strategy='most_frequent').fit_transform(df[column].values.reshape(-1, 1))[:, 0]
```

```
    print(df[column].value_counts())
```

```
region
columbus                   3608
jacksonville               3562
spokane / coeur d'alene    2988
eugene                     2985
fresno / madera            2983
                           ...
meridian                     28
southwest MS                 14
kansas city                  11
fort smith, AR                9
west virginia (old)           8
Name: count, Length: 404, dtype: int64
manufacturer
ford               88631
chevrolet          55064
toyota             34202
honda              21269
nissan             19067
jeep               19014
ram                18342
gmc                16785
bmw                14699
dodge              13707
mercedes-benz      11817
hyundai            10338
subaru              9495
volkswagen          9345
kia                 8457
lexus               8200
audi                7573
cadillac            6953
chrysler            6031
acura               5978
buick               5501
mazda               5427
infiniti            4802
lincoln             4220
volvo               3374
mitsubishi          3292
mini                2376
pontiac             2288
rover               2113
jaguar              1946
porsche             1384
mercury             1184
saturn              1090
```

```
alfa-romeo          897
tesla               868
fiat                792
harley-davidson     153
ferrari              95
datsun               63
aston-martin         24
land rover           21
morgan                3
```

One Hot Encoding Categorical Data

```
# One-Hot Encoding
import pandas as pd

# Identify categorical columns
categorical_columns = df.select_dtypes(include=['object']).columns
primary_columns = ['url', 'region_url', 'image_url', 'VIN']
categorical_columns = [column for column in categorical_columns if column not in primary_columns]

# Select categorical columns with up to 10 unique values
columns_to_encode_10_or_less = [col for col in categorical_columns if df[col].nunique() <= 10]

# Identify columns for top 10 most frequent values
columns_for_top_10 = ['manufacturer', 'type', 'region', 'title_status', 'transmission', 'drive', 'condition', 'fuel']
columns_to_encode_top_10 = []

# For all the columns, get up to the 10 most frequent values.
for col in columns_for_top_10:
    if col in categorical_columns:
        top_10_values = df[col].value_counts().nlargest(10).index.tolist()
        # Create new columns for each of the top 10 values
        for value in top_10_values:
            df[f'{col}_{value}'] = (df[col] == value).astype(int)
        columns_to_encode_top_10.append(col)


# Combine the lists of columns to encode
all_columns_to_encode = columns_to_encode_10_or_less + columns_to_encode_top_10

# Perform one-hot encoding on columns with 10 or fewer unique values
df_encoded = pd.get_dummies(df, columns=columns_to_encode_10_or_less, dummy_na=False, dtype=int)

# The top 10 columns have already been created as separate columns with boolean values.
# We can now drop the original 'manufacturer', 'type', 'state', and 'region' columns
df_encoded = df_encoded.drop(columns_for_top_10, axis=1, errors='ignore')


# Display the first few rows of the encoded DataFrame and its info to see the changes
```

```
display(df_encoded.head())
display(df_encoded.info())
```

	url	region_url	price	year	model	odometer	VIN	paint_color	image_url	description	...
0	https://prescott.craigslist.org/cto/d/prescott...	https://prescott.craigslist.org	6000	2013.0	f-150	85548.0	NaN	white	NaN	35 VEHICLES PRICED UNDER $3000!!! BIG TIME! T...	...
1	https://fayar.craigslist.org/ctd/d/bentonville...	https://fayar.craigslist.org	11900	2013.0	f-150	85548.0	NaN	white	NaN	35 VEHICLES PRICED UNDER $3000!!! BIG TIME! T...	...
2	https://keys.craigslist.org/cto/d/summerland-k...	https://keys.craigslist.org	21000	2013.0	f-150	85548.0	NaN	white	NaN	35 VEHICLES PRICED UNDER $3000!!! BIG TIME! T...	...
3	https://worcester.craigslist.org/cto/d/west-br...	https://worcester.craigslist.org	1500	2013.0	f-150	85548.0	NaN	white	NaN	35 VEHICLES PRICED UNDER $3000!!! BIG TIME! T...	...
4	https://greensboro.craigslist.org/cto/d/trinit...	https://greensboro.craigslist.org	4900	2013.0	f-150	85548.0	NaN	white	NaN	35 VEHICLES PRICED UNDER $3000!!! BIG TIME! T...	...

5 rows × 105 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 426880 entries, 0 to 426879
Columns: 105 entries, url to size_sub-compact
dtypes: float64(7), int64(89), object(9)
memory usage: 342.0+ MB
None
```

KDS Model Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class KDS(nn.Module):
```

```
    def __init__(
        self,
        num_layers,
        input_size,
        hidden_size,
        penalty,
        accelerate=True,
        train_step=True,
        W=None,
        step=None,
    ):
        super(KDS, self).__init__()

        # hyperparameters
        self.register_buffer("num_layers", torch.tensor(int(num_layers)))
        self.register_buffer("input_size", torch.tensor(int(input_size)))
        self.register_buffer("hidden_size", torch.tensor(int(hidden_size)))
        self.register_buffer("penalty", torch.tensor(float(penalty)))
        self.register_buffer("accelerate", torch.tensor(bool(accelerate)))

        # parameters
        if W is None:
            W = torch.zeros(self.hidden_size, self.input_size)
        self.register_parameter("W", torch.nn.Parameter(W))
        if step is None:
            step = W.svd().S[0] ** -2
        if train_step:
            self.register_parameter("step", torch.nn.Parameter(step))
        else:
            self.register_buffer("step", step)

    def forward(self, y):
        x = self.encode(y)
        y = self.decode(x)
        return y

    def encode(self, y):
        if self.accelerate:
            return self.encode_accelerated(y)
        else:
            return self.encode_basic(y)

    def encode_basic(self, y):
        x = torch.zeros(y.shape[0], self.hidden_size, device=y.device)
        # 'weight' here is actually distances_sq, used to compute gradient for x.
        distances_sq = (
            y.square().sum(dim=1, keepdims=True)
            + self.W.T.square().sum(dim=0, keepdims=True)
            - 2 * y @ self.W.T
        )
```

```
        for layer in range(self.num_layers):
            grad = (x @ self.W - y) @ self.W.T
            # The penalty here influences the sparsity of x during the encoding process
            grad = grad + distances_sq * self.penalty
            x = self.activate(x - grad * self.step)
        return x

    def encode_accelerated(self, y):
        x_tmp = torch.zeros(y.shape[0], self.hidden_size, device=y.device)
        x_old = torch.zeros(y.shape[0], self.hidden_size, device=y.device)
        # 'weight' here is actually distances_sq, used to compute gradient for x.
        distances_sq = (
            y.square().sum(dim=1, keepdims=True)
            + self.W.T.square().sum(dim=0, keepdims=True)
            - 2 * y @ self.W.T
        )
        for layer in range(self.num_layers):
            grad = (x_tmp @ self.W - y) @ self.W.T
            # The penalty here influences the sparsity of x during the encoding process
            grad = grad + distances_sq * self.penalty
            x_new = self.activate(x_tmp - grad * self.step)
            x_old, x_tmp = x_new, x_new + layer / (layer + 3) * (x_new - x_old)
        return x_new

    def decode(self, x):
        return x @ self.W

    def activate(self, x):
        m, n = x.shape
        cnt_m = torch.arange(m, device=x.device)
        cnt_n = torch.arange(n, device=x.device)
        u = x.sort(dim=1, descending=True).values
        v = (u.cumsum(dim=1) - 1) / (cnt_n + 1)
        w = v[cnt_m, (u > v).sum(dim=1) - 1]
        return (x - w.view(m, 1)).relu()
```

```
# Loss
class LocalDictionaryLoss(torch.nn.Module):
    def __init__(self, penalty):
        super(LocalDictionaryLoss, self).__init__()
        self.penalty = penalty

    def forward(self, A, y, x):
        return self.forward_detailed(A, y, x)[2]

    def forward_detailed(self, A, y, x):
        weight = (y.unsqueeze(1) - A.unsqueeze(0)).pow(2).sum(dim=2)
        a = 0.5 * (y - x @ A).pow(2).sum(dim=1).mean()
        b = (weight * x).sum(dim=1).mean()
        return a, b, a + b * self.penalty
```

Data Preparation

```
# Preparing the data, extracting the prices.
from tqdm import tqdm
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, QuantileTransformer
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

def prepare_data(df, feature_cols, test_size=0.3, val_size=0.15):
    """
    Prepare data for training

    Args:
        df: DataFrame with features and price
        feature_cols: List of feature column names
        test_size: Fraction for test set
        val_size: Fraction of train for validation

    Returns:
        Dictionary with train/val/test splits and scaler
    """
    # X will contain features specified in feature_cols
    X = df[feature_cols].values
    print(f"[prepare_data] Shape of X (before split): {X.shape}") # Diagnostic print
    print(f"[prepare_data] X features head (first row): {X[0, :5]}...") # Diagnostic print

    # y is the target price, log1p transformed for prediction
    y = df['price'].values

    # Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42
```

```
        )

        # Train/val split
        X_train, X_val, y_train, y_val = train_test_split(
            X_train, y_train, test_size=val_size, random_state=42
        )

        print(f"X_train min/max BEFORE feature scaling: [{X_train.min():.2f}, {X_train.max():.2f}]")
        # This is more robust to outliers than StandardScaler or MinMaxScaler for non-Gaussian data
        feature_scaler = QuantileTransformer(output_distribution='uniform', random_state=42)
        print(f"Type of feature_scaler: {type(feature_scaler)}")
        X_train = feature_scaler.fit_transform(X_train)
        X_val = feature_scaler.transform(X_val)
        X_test = feature_scaler.transform(X_test)
        print(f"X_train min/max AFTER feature scaling: [{X_train.min():.2f}, {X_train.max():.2f}]")

        price_scaler = QuantileTransformer(output_distribution='uniform', random_state=42)
        y_train_scaled = price_scaler.fit_transform(y_train.reshape(-1, 1))
        y_val_scaled = price_scaler.transform(y_val.reshape(-1, 1))
        y_test_scaled = price_scaler.transform(y_test.reshape(-1, 1))

        print(f"Train: {len(X_train)} samples")
        print(f"Val: {len(X_val)} samples")
        print(f"Test: {len(X_test)} samples")
        print(f"Feature dim: {X_train.shape[1]}")
        # Print transformed price range and stats
        print(f"Transformed Price range (log1p): {y_train_scaled.min():.2f} - {y_train_scaled.max():.2f}")
        print(f"Transformed Price mean (log1p): {y_train_scaled.mean():.2f} \u00b1 {y_train_scaled.std():.2f}")

        print(f"\u2713 Features range: [{X_train.min():.2f}, {X_train.max():.2f}]")
        print(f"\u2713 Prices range: [{y_train_scaled.min():.2f}, {y_train_scaled.max():.2f}]")

        return {
            'X_train': X_train,
            'y_train': y_train_scaled,
            'X_val': X_val,
            'y_val': y_val_scaled,
            'X_test': X_test,
            'y_test': y_test_scaled,
            'feature_scaler': feature_scaler,
            'price_scaler': price_scaler,
            'y_train_raw': y_train,
            'y_val_raw': y_val,
            'y_test_raw': y_test,
        }

    epsilon = 1e-6
    df_encoded['price_per_mileage'] = df_encoded['price'] / (df_encoded['odometer'] + epsilon)
    df_encoded['mileage_per_year'] = df_encoded['odometer'] / (df_encoded['age'] + epsilon)
```

```
# --- MODIFIED: feature_columns now explicitly EXCLUDES 'price', 'price_per_mileage', and 'mileage_per_year' ---
feature_columns = [col for col in df_encoded.select_dtypes(include=['number']).columns if col not in ['price', 'price_per_mileage', 'mi

print(f"Feature columns for KDS model: {feature_columns}")
print(f"Length of feature_columns: {len(feature_columns)}") # Diagnostic print

# Diagnostic: Check df_encoded numeric columns
print("\n--- df_encoded Numeric Column Check ---")
print(f"df_encoded.shape: {df_encoded.shape}")
numeric_cols_in_df_encoded = df_encoded.select_dtypes(include=['number']).columns.tolist()
print(f"Numeric columns in df_encoded: {len(numeric_cols_in_df_encoded)}")
print(f"Numeric column names (first 5): {numeric_cols_in_df_encoded[:5]}")
print(f"'price' column in df_encoded numeric columns: {'price' in numeric_cols_in_df_encoded}")
print("--------------------------------------")

data_splits = prepare_data(df_encoded, feature_columns)

# Unpack the dictionary into individual variables
X_train = data_splits['X_train']
y_train = data_splits['y_train']
X_val = data_splits['X_val']
y_val = data_splits['y_val']
X_test = data_splits['X_test']
y_test = data_splits['y_test']
feature_scaler = data_splits['feature_scaler']
price_scaler = data_splits['price_scaler']
y_train_raw = data_splits['y_train_raw']
y_val_raw = data_splits['y_val_raw']
y_test_raw = data_splits['y_test_raw']
```

```
Feature columns for KDS model: ['year', 'odometer', 'lat', 'long', 'age', 'manufacturer_ford', 'manufacturer_chevrolet', 'manufacturer_t
Length of feature_columns: 93

--- df_encoded Numeric Column Check ---
df_encoded.shape: (426880, 105)
Numeric columns in df_encoded: 96
Numeric column names (first 5): ['price', 'year', 'odometer', 'lat', 'long']
'price' column in df_encoded numeric columns: True
--------------------------------------
[prepare_data] Shape of X (before split): (426880, 139)
[prepare_data] X features head (first row): [ 2.01300e+03  8.55480e+04  3.91501e+01 -8.84326e+01  1.20000e+01]...
X_train min/max BEFORE feature scaling: [-159.72, 10000000.00]
Type of feature_scaler: <class 'sklearn.preprocessing._data.QuantileTransformer'>
X_train min/max AFTER feature scaling: [0.00, 1.00]
Train: 253993 samples
Val: 44823 samples
Test: 128064 samples
Feature dim: 139
Transformed Price range (log1p): 0.00 - 1.00
Transformed Price mean (log1p): 0.50 ± 0.29
✓ Features range: [0.00, 1.00]
```

```
✓ Prices range: [0.00, 1.00]
```

```
# Random Sampling of the data
import numpy as np
import numpy.random as random
import torch

# Set seeds for reproducibility
random.seed(42)
torch.manual_seed(42) # Set torch seed

# Diagnostic: Check X_train shape immediately before sampling
print(f"[Sampling Check] X_train shape BEFORE sampling: {X_train.shape}")

n_samples = 10000
random_indices = random.choice(len(X_train), n_samples, replace=False)
X_reduced_train = X_train[random_indices]
y_reduced_train = y_train[random_indices] # Add this line to sample y_train
print(f"[Sampling] Shape of X_reduced_train: {X_reduced_train.shape}") # Diagnostic print
print(f"[Sampling] Shape of y_reduced_train: {y_reduced_train.shape}") # Diagnostic print

n_test_samples = 5000
random_test_indices = random.choice(len(X_test), n_test_samples, replace=False)
X_reduced_test = X_test[random_test_indices]
y_reduced_test = y_test[random_test_indices] # Add this line to sample y_test
print(f"[Sampling] Shape of X_reduced_test: {X_reduced_test.shape}") # Diagnostic print
print(f"[Sampling] Shape of y_reduced_test: {y_reduced_test.shape}") # Diagnostic print

# Random validation data, convert to Tensors
n_val_samples = 3000
random_val_indices = np.random.choice(len(X_val), n_val_samples, replace=False)
X_reduced_val = X_val[random_val_indices]
y_reduced_val = y_val[random_val_indices] # Add this line to sample y_val
print(f"[Sampling] Shape of X_reduced_val: {X_reduced_val.shape}") # Diagnostic print
print(f"[Sampling] Shape of y_reduced_val: {y_reduced_val.shape}") # Diagnostic print
```

```
[Sampling Check] X_train shape BEFORE sampling: (253993, 139)
[Sampling] Shape of X_reduced_train: (10000, 139)
[Sampling] Shape of y_reduced_train: (10000, 1)
[Sampling] Shape of X_reduced_test: (5000, 139)
[Sampling] Shape of y_reduced_test: (5000, 1)
[Sampling] Shape of X_reduced_val: (3000, 139)
[Sampling] Shape of y_reduced_val: (3000, 1)
```

```
print(df_encoded.info)
```

```
<bound method DataFrame.info of                                                   url  \
0       https://prescott.craigslist.org/cto/d/prescott...
1       https://fayar.craigslist.org/ctd/d/bentonville...
2       https://keys.craigslist.org/cto/d/summerland-k...
```

```
3       https://worcester.craigslist.org/cto/d/west-br...
4       https://greensboro.craigslist.org/cto/d/trinit...
...                                                  ...
426875  https://wyoming.craigslist.org/ctd/d/atlanta-2...
426876  https://wyoming.craigslist.org/ctd/d/atlanta-2...
426877  https://wyoming.craigslist.org/ctd/d/atlanta-2...
426878  https://wyoming.craigslist.org/ctd/d/atlanta-2...
426879  https://wyoming.craigslist.org/ctd/d/atlanta-2...

                              region_url  price    year  \
0         https://prescott.craigslist.org   6000  2013.0
1            https://fayar.craigslist.org  11900  2013.0
2             https://keys.craigslist.org  21000  2013.0
3        https://worcester.craigslist.org   1500  2013.0
4       https://greensboro.craigslist.org   4900  2013.0
...                                   ...    ...     ...
426875     https://wyoming.craigslist.org  23590  2019.0
426876     https://wyoming.craigslist.org  30590  2020.0
426877     https://wyoming.craigslist.org  34990  2020.0
426878     https://wyoming.craigslist.org  28990  2018.0
426879     https://wyoming.craigslist.org  30590  2019.0

                          model  odometer                VIN paint_color  \
0                         f-150   85548.0                NaN       white
1                         f-150   85548.0                NaN       white
2                         f-150   85548.0                NaN       white
3                         f-150   85548.0                NaN       white
4                         f-150   85548.0                NaN       white
...                         ...       ...                ...         ...
426875         maxima s sedan 4d   32226.0  1N4AA6AV6KC367801       white
426876  s60 t5 momentum sedan 4d   12029.0  7JR102FKXLG042696         red
426877          xt4 sport suv 4d    4174.0  1GYFZFR46LF088296       white
426878           es 350 sedan 4d   30112.0  58ABK1GG4JU103853      silver
426879  4 series 430i gran coupe   22716.0  WBA4J1C58KBM14708       white

                                             image_url  \
0                                                  NaN
1                                                  NaN
2                                                  NaN
3                                                  NaN
4                                                  NaN
...                                                ...
426875  https://images.craigslist.org/00o0o_iiraFnHg8q...
426876  https://images.craigslist.org/00x0x_15sbgnxCIS...
426877  https://images.craigslist.org/00L0L_farM7bxnxR...
426878  https://images.craigslist.org/00z0z_bKnIVGLkDT...
426879  https://images.craigslist.org/00Y0Y_lEUocjyRxa...

                                           description  ...  \
0       35 VEHICLES PRICED UNDER $3000!!!  BIG TIME! T...  ...
1       35 VEHICLES PRICED UNDER $3000!!!  BIG TIME! T...  ...
2       35 VEHICLES PRICED UNDER $3000!!!  BIG TIME! T...  ...
3       35 VEHICLES PRICED UNDER $3000!!!  BIG TIME! T...  ...
4       35 VEHICLES PRICED UNDER $3000!!!  BIG TIME! T...  ...
```

Optimal Number of Clusters to Establish Hidden Size

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

random_state = 42
# Elbow Plot Method
k_rng = range(1, 501, 25)
sse = []

# Compute WCSE for all possible total clusters.
for k in k_rng:
  km = KMeans(n_clusters=k, random_state=random_state)
  km.fit(X_reduced_train)
  sse.append(km.inertia_)
  print(f'K value {k} + inertia {km.inertia_}')

# Plot the Elbow Plot
plt.plot(k_rng, sse, linestyle='--', marker='o', color='black')
plt.xlabel("Number of Clusters (k)")
plt.ylabel('WCSE')

plt.plot
```

```
K value 1 + inertia 93587.63621851073
K value 26 + inertia 30534.577613219244
K value 51 + inertia 23780.486440993733
K value 76 + inertia 21025.249282775265
K value 101 + inertia 18585.5567614129
```

From the visual, we notice a significantly sharp decline when the number of clusters k = 51. We use 51 clusters from here on out moving forward.

```
# Set up the chosen K Means
hidden_size = 90
km = KMeans(n_clusters=hidden_size, random_state=random_state)
km.fit(X_reduced_train, y_reduced_train)

# Take the clustering centers
A = km.cluster_centers_

# Get the pseudo archetype information
print(f"Pseudo Archetype (A) : {A.shape}")
print(f'Weighted Cluster Sum of Square Errors - {km.inertia_}')
```

```
K value 476 + inertia 9807.515824066446
Pseudo Archetype (A) : (90, 139)
Weighted Cluster Sum of Square Errors - 19647.930919571838
```

matplotlib.pyplot.plot

```
def plot(*args: float | ArrayLike | str, scalex: bool=True, scaley: bool=True, data=None,
**kwargs) -> list[Line2D]
```

/usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py

Plot y versus x as lines and/or markers.

Sample Random Probability Vectors Test Code

```
# Kraemer Algorithm for sampling random probability vectors on the simplex.
# Sum for i=1 to n, p_i >= 0
# Sum of the total probabilities must be equal to 1.

def sample_simplex_uniform(n_samples, n_dim, random_state = None):
  '''
  Sample random probability vectors on the simplex using the Kraemer Algorithm.

  Algorithm Steps:
  1. Select x_1,.....,x_(n-1) from a uniform distribution between 0 to 1.
  2. Sort the x_i in place.
  3. Compute the differences between the consecutive values.

  Args:
      n_samples: Number of probability vectors to generate
      n_dim: Dimension of each probability vector (hidden_size)
      random_state: Random seed for reproducibility

  Returns:
      codes: (n_samples, n_dim) array where each row sums to 1
  '''

  if random_state is not None:
    np.random.seed(random_state)
```

```
    # Step 1: Select the n-1 x values uniformly distribution between 0 to 1.
    samples_uniform = np.random.uniform(0, 1, size=(n_samples, n_dim - 1))

    # Achieve boundaries, sort them
    with_boundaries = np.concatenate(
        [np.zeros((n_samples, 1)),
         samples_uniform,
         np.ones((n_samples, 1))]
    , axis=1)

    # Step 2: Sort the x_is in place
    print(with_boundaries)
    sorted_values = np.sort(with_boundaries, axis=1)

    # Step 3: Take the differences between consecutive sorted values
    codes = np.diff(sorted_values, axis=1)
    assert np.allclose(codes.sum(axis=1), 1.0)

    return codes
```

```
# Generate the random codes for the training points.
n_train = X_reduced_train.shape[0]
random_codes = sample_simplex_uniform(
    n_samples=n_train,
    n_dim=hidden_size,
    random_state=45
)

# Print statements to test the sample simplex uniform functionality.
print(f'Random codes shape: {random_codes.shape}')
print(f'Codes sum to 1: {np.allclose(random_codes.sum(axis=1), 1.0)}')
print(f'All non-negative: {(random_codes >= 0).all()}')
print(f'Example code (first point): {random_codes[0, :5]}... (showing first 5)')
print(f'Example code sum: {random_codes[0].sum():.6f}')
print(f'Mean sparsity: {(random_codes > 0.01).sum(axis=1).mean():.2f} active components')
```

```
[[0.         0.98901151 0.54954473 ... 0.7267484  0.25263225 1.        ]
 [0.         0.9074539  0.68472269 ... 0.12543415 0.80652074 1.        ]
 [0.         0.87720115 0.58917432 ... 0.95365309 0.72979648 1.        ]
 ...
 [0.         0.54655248 0.35956076 ... 0.4102572  0.43958778 1.        ]
 [0.         0.09713954 0.68466191 ... 0.05604839 0.08285445 1.        ]
 [0.         0.55084116 0.10777903 ... 0.19800531 0.33108688 1.        ]]
Random codes shape: (10000, 90)
Codes sum to 1: True
All non-negative: True
Example code (first point): [3.94198029e-02 9.10219802e-03 1.56502951e-02 5.98754738e-05
 2.70609975e-03]... (showing first 5)
Example code sum: 1.000000
Mean sparsity: 36.79 active components
```

Compute the Reconstruction to Locality Loss Ratio

```
def compute_loss_ratio(X, A, codes, batch_size=1000):
    """
    Compute the ratio of reconstruction loss to locality loss (memory-efficient).

    Args:
        X: (n_samples, input_size) data matrix
        A: (hidden_size, input_size) pseudo-dictionary (archetypes)
        codes: (n_samples, hidden_size) random probability vectors
        batch_size: Process this many samples at a time

    Returns:
        ratio: reconstruction_loss / locality_loss
        recon_loss: reconstruction loss value
        locality_loss: locality loss value
    """
    n_samples = X.shape[0]

    total_recon_loss = 0.0
    total_locality_loss = 0.0

    # Process in batches to save memory
    for start_idx in range(0, n_samples, batch_size):
        end_idx = min(start_idx + batch_size, n_samples)

        X_batch = X[start_idx:end_idx]
        codes_batch = codes[start_idx:end_idx]
        batch_n = X_batch.shape[0]

        # Reconstruction: y_hat = codes @ A
        Y_hat_batch = codes_batch @ A

        # Reconstruction loss for this batch
        recon_batch = 0.5 * np.sum((X_batch - Y_hat_batch) ** 2)
        total_recon_loss += recon_batch

        # Locality loss: sum_j x_ij * ||y_i - a_j||^2
        # Compute using efficient matrix operations
        # distances_sq[i,j] = ||X_batch[i] - A[j]||^2

        # Method: ||a - b||^2 = ||a||^2 + ||b||^2 - 2*a·b
        X_sq = np.sum(X_batch ** 2, axis=1, keepdims=True)  # (batch_n, 1)
        A_sq = np.sum(A ** 2, axis=1, keepdims=True).T      # (1, hidden_size)
        XA = X_batch @ A.T                                    # (batch_n, hidden_size)

        distances_sq = X_sq + A_sq - 2 * XA  # (batch_n, hidden_size)

        # Locality loss for this batch
```

```
        locality_batch = np.sum(codes_batch * distances_sq)
        total_locality_loss += locality_batch

        # Optional: Print progress for large datasets
        if (end_idx % 50000) == 0:
            print(f'  Processed {end_idx}/{n_samples} samples...')

    # Average over all samples
    reconstruction_loss = total_recon_loss / n_samples
    locality_loss = total_locality_loss / n_samples

    # Compute ratio
    ratio = reconstruction_loss / locality_loss if locality_loss > 0 else np.inf

    return ratio, reconstruction_loss, locality_loss


# Step 3: Compute loss ratios (memory-efficient)
print("\nStep 3: Computing loss ratios with random codes...")

# You can now use the full training set
ratio, recon, locality = compute_loss_ratio(
    X_reduced_train,
    A,
    random_codes,
    batch_size=2000  # Adjust based on your memory
)

print(f'\nResults on {X_train.shape[0]} samples:')
print(f'Reconstruction loss: {recon:.6f}')
print(f'Locality loss: {locality:.6f}')
print(f'Ratio (recon/locality): {ratio:.6f}')
print(f'\nThis means locality is naturally {1/ratio:.2f}x larger than reconstruction')
print(f'Suggested C value (to balance): {ratio:.6f}')
```

```
Step 3: Computing loss ratios with random codes...

Results on 253993 samples:
Reconstruction loss: 4.814086
Locality loss: 18.386916
Ratio (recon/locality): 0.261821

This means locality is naturally 3.82x larger than reconstruction
Suggested C value (to balance): 0.261821
```

Run the experiment with multiple trials

```
# Multiple trials
def estimate_C_multiple_trials(X_train, hidden_size, n_trials=10, sample_size=10000, batch_size=2000, random_state = None):
```

```
  """
    Estimate C by running multiple trials with different:
    - K-means initializations
    - Random code samples

    Args:
        X_train: Training data (scaled)
        hidden_size: Number of archetypes/clusters
        n_trials: Number of independent trials
        sample_size: Number of data points to use per trial (for speed)
        batch_size: Batch size for loss computation

    Returns:
        ratios: List of ratios from each trial
        mean_C: Average C estimate
        std_C: Standard deviation of C estimates
    """

  if random_state is not None:
    np.random.seed(random_state)

  ratios = []
  for trial in range(n_trials):
    print(f'\n --- Trial {trial + 1} / {n_trials} ---')

    # Sample random indices of my data
    indices = np.random.choice(X_train.shape[0], size=sample_size, replace=False)
    X_sample = X_train[indices]

    # Step 1: K-Means with different initializations
    km = KMeans(n_clusters=hidden_size, random_state=trial, n_init=10)
    km.fit(X_sample)
    A = km.cluster_centers_
    print(f"Pseudo Archetype (A) : {A.shape}")
    print(f'Weighted Cluster Sum of Square Errors for trial {trial} - {km.inertia_}')

    # Step 2: Random sparse codes for each trial
    random_codes = sample_simplex_uniform(
        n_samples=sample_size,
        n_dim=hidden_size,
        random_state=trial * 10
    )

    # Step 3: Compute the loss ratio
    # You can now use the full training set
    ratio, recon, locality = compute_loss_ratio(
        X_sample,
        A,
        random_codes,
        batch_size=2000  # Adjust based on your memory
```

```
    )

    print(f'\n')
    print(f'Computed Ratio: {ratio:.6f}')
    print(f'Computed Recon: {recon:.6f}')
    print(f'Computed Locality: {locality:.6f}')

    ratios.append(ratio)

  ratios = np.array(ratios)
  mean_C_value = np.mean(ratios)
  std_C_value = np.std(ratios)

  return ratios, mean_C_value, std_C_value
```

```
# Run the multiple trials
ratios, mean_C, std_C = estimate_C_multiple_trials(
    X_train=X_train,
    hidden_size=hidden_size,
    n_trials=15,
    sample_size=10000,
    batch_size=2000,
    random_state=5
)

print(f'{"="*10}')
print("FINAL C ESTIMATION RESULTS")
print(f'Ratios across trials: {ratios}')
print(f'Mean C: {mean_C:.6f} ± {std_C:.6f}')
print(f'{"="*10}')
```

```
 --- Trial 1 / 15 ---
Pseudo Archetype (A) : (90, 139)
Weighted Cluster Sum of Square Errors for trial 0 - 19139.258679471066
[[0.         0.5488135  0.71518937 ... 0.09394051 0.5759465  1.        ]
 [0.         0.9292962  0.31856895 ... 0.34535168 0.92808129 1.        ]
 [0.         0.7044144  0.03183893 ... 0.78515291 0.28173011 1.        ]
 ...
 [0.         0.16194469 0.12714128 ... 0.39469482 0.85190647 1.        ]
 [0.         0.24997749 0.03327364 ... 0.46113044 0.66641497 1.        ]
 [0.         0.77604911 0.61176266 ... 0.17781237 0.82181806 1.        ]]


Computed Ratio: 0.261424
Computed Recon: 4.832254
Computed Locality: 18.484327

 --- Trial 2 / 15 ---
Pseudo Archetype (A) : (90, 139)
```

```
Weighted Cluster Sum of Square Errors for trial 1 - 19347.441703832486
[[0.         0.77132064 0.02075195 ... 0.24207588 0.55757819 1.        ]
 [0.         0.56550702 0.47513225 ... 0.75046461 0.17604213 1.        ]
 [0.         0.45851421 0.51312271 ... 0.19325373 0.04673923 1.        ]
 ...
 [0.         0.18217918 0.04789739 ... 0.62394208 0.08421872 1.        ]
 [0.         0.80170923 0.26267496 ... 0.37481094 0.47330771 1.        ]
 [0.         0.74066582 0.17684519 ... 0.87989411 0.86535433 1.        ]]


Computed Ratio: 0.259529
Computed Recon: 4.833747
Computed Locality: 18.625111

 --- Trial 3 / 15 ---
Pseudo Archetype (A) : (90, 139)
Weighted Cluster Sum of Square Errors for trial 2 - 19470.622121640437
[[0.         0.5881308  0.89771373 ... 0.28402501 0.10760726 1.        ]
 [0.         0.11700177 0.7555236  ... 0.68952736 0.76848649 1.        ]
 [0.         0.44285459 0.95131449 ... 0.89017894 0.73103431 1.        ]
 ...
 [0.         0.21507613 0.70222423 ... 0.01589255 0.83442366 1.        ]
 [0.         0.53267315 0.29815329 ... 0.81538336 0.23034099 1.        ]
 [0.         0.89203724 0.89861977 ... 0.98723949 0.70928011 1.        ]]


Computed Ratio: 0.259250
Computed Recon: 4.878619
Computed Locality: 18.818208

 --- Trial 4 / 15 ---
Pseudo Archetype (A) : (90, 139)
Weighted Cluster Sum of Square Errors for trial 3 - 19916.125927031455
[[0.         0.64414354 0.38074849 ... 0.36849024 0.30672666 1.        ]
 [0.         0.34086324 0.29047775 ... 0.19520153 0.55108057 1.        ]
 [0.         0.54861257 0.71287215 ... 0.16657218 0.26559834 1.        ]
 ...
 [0.         0.17420712 0.91992561 ... 0.0740423  0.06384866 1.        ]
```

Training of the KDS Model.

```
import torch
from torch.utils.data import TensorDataset, DataLoader
import random
import tensorflow as tf
import torch
import numpy as np
import torch.optim as optim

# Set seeds for reproducibility
random.seed(42)
np.random.seed(42)
```

```
tf.random.set_seed(42)
torch.manual_seed(42) # Set torch seed

# Convert numpy arrays to PyTorch tensors
X_train_tensor = torch.tensor(X_reduced_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_reduced_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_reduced_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_reduced_test, dtype=torch.float32)
X_val_tensor = torch.tensor(X_reduced_val, dtype=torch.float32)
y_val_tensor = torch.tensor(y_reduced_val, dtype=torch.float32)

# Create Tensor Datasets
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)

# Create Data Loaders
batch_size = 32  # You can adjust the batch size
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)

val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
```

```
# Define the base path again to ensure it's correct
import os
base_path = '/content/drive/My Drive/Tufts/Masters Thesis Research/Car_Models_Completed_Numeric_Categorical_90_hidden_layers_15_encoding

# Check if the directory exists
if os.path.exists(base_path):
    print(f"Directory '{base_path}' exists.")
    print("Contents of the directory:")
    for item in os.listdir(base_path):
        print(f"- {item}")
else:
    print(f"Directory '{base_path}' DOES NOT exist. Please ensure the path is correct and your Drive is mounted properly.")
```

```
Directory '/content/drive/My Drive/Tufts/Masters Thesis Research/Car_Models_Completed_Numeric_Categorical_90_hidden_layers_15_encoding_la
Contents of the directory:
- kds_model_0.005x sparse.pth
- kds_model_0.001x sparse.pth
- kds_model_0.00075x sparse.pth
- kds_model_0.00025x sparse.pth
- kds_model_0.0001x sparse.pth
- kds_model_0.00001x sparse.pth
- kds_model_0.000005x sparse.pth
```

```
from tqdm import tqdm as progress_bar

input_size_features = X_train.shape[1] # This should now be 35
```

```
    print(f"[Model Init] input_size_features (from X_train.shape[1]): {input_size_features}") # Diagnostic print

    hidden_size = hidden_size  # You can adjust the number of archetypes
    num_layers = 15 # You can adjust the number of encoding layers
    t_parameters = {
        '0.0001x sparse': (mean_C * 0.0001) / mean_C,
        '0.00001x sparse': (mean_C * 0.00001) / mean_C,
        '0.000005x sparse': (mean_C * 0.000005) / mean_C,
    }

    # Get the K-means cluster centers for W initialization (from previous steps)
    # Ensure A is a tensor and on the correct device
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Re-run K-means to get A based on X_reduced_train (which now has 37 features)
    km = KMeans(n_clusters=hidden_size, random_state=random_state) # Use random_state from earlier
    km.fit(X_reduced_train) # Fit on X_reduced_train which now has 37 features
    A = km.cluster_centers_

    print(f"[Model Init] Shape of A (after re-running KMeans): {A.shape}") # Diagnostic print

    # A now has shape (hidden_size, input_size_features) i.e. (51, 37)
    initial_W = torch.tensor(A, dtype=torch.float32).to(device)

    num_epochs = 100

    for i in range(len(t_parameters)):
      cur_key = list(t_parameters.keys())[i]
      cur_t = list(t_parameters.values())[i]

      # Instantiate the model with the correct input_size (37) and no price_column_index
      net = KDS(
          num_layers=num_layers,
          input_size=input_size_features, # Use the input_size_features (37)
          hidden_size=hidden_size,
          penalty=mean_C * cur_t, # Use the dynamic penalty from t_parameters
          accelerate=True,
          train_step=True,
          W=initial_W # Pass the K-means cluster centers as initial W,
      ).to(device)

      # Diagnostic prints to check the model object
      print(f"Type of 'model' before training loop: {type(net)}")
      print(f"Is 'model' an instance of KDS? {isinstance(net, KDS)}")
      print(f"Is 'model' an instance of torch.nn.Module? {isinstance(net, torch.nn.Module)}")
      print(f"[Model Init] Model's W shape: {net.W.shape}") # Diagnostic print

      # Define loss function and optimizer
      criterion = LocalDictionaryLoss(penalty=mean_C * cur_t)
      learning_rate = 1e-3
```

```
num_epochs = 100 # This can be tuned
optimizer = optim.Adam(net.parameters(), lr=0.001) # You can adjust the learning rate. Try 1e-4.

train_losses = []
val_losses = []

print(f"Starting training for {num_epochs} epochs...")
print(f'----- Cur Key Step: {cur_key}, Cur T Value: {cur_t} ------')

with torch.no_grad():
    p = torch.randperm(len(train_dataset))[: net.hidden_size]
    net.W.data = X_train_tensor[p]
    net.step.fill_((net.W.data.svd()[1][0] ** -2).item())
net = net.to(device)

for epoch in progress_bar(range(num_epochs)):
    shuffle = torch.randperm(len(train_dataset))
    data, labels = X_train_tensor[shuffle], y_train_tensor[shuffle]
    for i in progress_bar(range(0, len(data), 32), disable=True):
        y = data[i : i + 32].to(device)
        x_hat = net.encode(y)
        loss = criterion(net.W, y, x_hat)
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(net.parameters(), 1e-4)
        optimizer.step()

    if epoch + 1 % 10 == 0:
        print(f"Epoch {epoch + 1}/{num_epochs}, Loss: {loss.item():.4f}")

print("\n\nTraining finished.")

file_name = f'kds_model_{cur_key}.pth'
path = os.path.join(base_path, file_name)
torch.save(net.state_dict(), path)

# Create the directory if it does not exist
os.makedirs(base_path, exist_ok=True)
```

```
[Model Init] input_size_features (from X_train.shape[1]): 139
[Model Init] Shape of A (after re-running KMeans): (90, 139)
Type of 'model' before training loop: <class '__main__.KDS'>
Is 'model' an instance of KDS? True
Is 'model' an instance of torch.nn.Module? True
[Model Init] Model's W shape: torch.Size([90, 139])
Starting training for 100 epochs...
----- Cur Key Step: 0.0001x sparse, Cur T Value: 0.0001 ------
 23%|██        | 23/100 [02:57<09:53,  7.71s/it]
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipython-input-3692045616.py in <cell line: 0>()
     74         x_hat = net.encode(y)
     75         loss = criterion(net.W, y, x_hat)
---> 76         optimizer.zero_grad()
     77         loss.backward()
     78         nn.utils.clip_grad_norm_(net.parameters(), 1e-4)

/usr/local/lib/python3.12/dist-packages/torch/_compile.py in inner(*args, **kwargs)
     39     if fn is not None:
     40 
---> 41         @functools.wraps(fn)
     42         def inner(*args: _P.args, **kwargs: _P.kwargs) -> _T:
     43             # cache this on the first invocation to avoid adding too much overhead.

KeyboardInterrupt:
```

Model Evaluation and Synthesis

Load trained models from the disk

```
input_size_features = X_train.shape[1] # This should now be 37
print(f"[Model Init] input_size_features (from X_train.shape[1]): {input_size_features}") # Diagnostic print

hidden_size = hidden_size  # You can adjust the number of archetypes
num_layers = 15 # You can adjust the number of encoding layers
t_parameters = {
    '0.0001x sparse': (mean_C * 0.0001) / mean_C,
    '0.00001x sparse': (mean_C * 0.00001) / mean_C,
    '0.000005x sparse': (mean_C * 0.000005) / mean_C,
}
```

```
[Model Init] input_size_features (from X_train.shape[1]): 139
```

```
kds_model_names = ['0.0001x sparse', '0.00001x sparse', '0.000005x sparse']
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

kds_models = []
```

```
for i in range(len(kds_model_names)):
  cur_key = kds_model_names[i]
  cur_t = t_parameters[cur_key]

  # Re-run K-means to get A based on X_reduced_train (which now has 37 features)
  km = KMeans(n_clusters=hidden_size, random_state=random_state) # Use random_state from earlier
  km.fit(X_reduced_train) # Fit on X_reduced_train which now has 37 features
  A = km.cluster_centers_

  print(f"[Model Init] Shape of A (after re-running KMeans): {A.shape}") # Diagnostic print

  # A now has shape (hidden_size, input_size_features) i.e. (51, 37)
  initial_W = torch.tensor(A, dtype=torch.float32).to(device)

  net = KDS(
    num_layers=num_layers,
    input_size=input_size_features, # Use the input_size_features (37)
    hidden_size=hidden_size,
    penalty=mean_C * cur_t, # Use the dynamic penalty from t_parameters
    accelerate=True,
    train_step=True,
    W=initial_W # Pass the K-means cluster centers as initial W,
  ).to(device)

  full_pth = f'{base_path}/kds_model_{kds_model_names[i]}.pth'
  print(full_pth)
  net.load_state_dict(torch.load(full_pth))

  # Move the model to the device (e.g., GPU if available) if it was trained on it
  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
  net = net.to(device)

  kds_models.append(net)
```

```
[Model Init] Shape of A (after re-running KMeans): (90, 139)
/content/drive/My Drive/Tufts/Masters Thesis Research/Car_Models_Completed_Numeric_Categorical_90_hidden_layers_15_encoding_layers/kds_mo
[Model Init] Shape of A (after re-running KMeans): (90, 139)
/content/drive/My Drive/Tufts/Masters Thesis Research/Car_Models_Completed_Numeric_Categorical_90_hidden_layers_15_encoding_layers/kds_mo
[Model Init] Shape of A (after re-running KMeans): (90, 139)
/content/drive/My Drive/Tufts/Masters Thesis Research/Car_Models_Completed_Numeric_Categorical_90_hidden_layers_15_encoding_layers/kds_mo
```

```
# Get the sparse codes for all the models with the data points and the atoms.
model_sparse_codes = {}
individual_sparse_codes = []
for i in range(len(kds_model_names)):
  print(f"Model Name: {kds_model_names[i]}")
  model = kds_models[i]
  with torch.no_grad():
    y = torch.tensor(X_reduced_train, dtype=torch.float32).to(device)
```

```
        sparse_codes = model.encode(y)
        sparse_codes = sparse_codes.cpu().numpy()
        individual_sparse_codes.append(sparse_codes)
        model_sparse_codes[kds_model_names[i]] = sparse_codes

print(model_sparse_codes.keys())
```

```
Model Name: 0.0001x sparse
Model Name: 0.00001x sparse
Model Name: 0.000005x sparse
dict_keys(['0.0001x sparse', '0.00001x sparse', '0.000005x sparse'])
```

```
def analyze_sparsity(sparse_codes, tolerance=1e-5):
    """
    Analyze the sparsity pattern of KDS sparse codes

    Args:
        sparse_codes: (n_samples, hidden_size) array of sparse codes
        tolerance: Threshold below which values are considered zero

    Returns:
        Dictionary with sparsity statistics
    """
    print("="*70)
    print("TASK 1: SPARSITY ANALYSIS")
    print("="*70)

    # Apply tolerance threshold
    sparse_codes_binary = sparse_codes > tolerance

    # Count active components per sample
    active_per_sample = np.sum(sparse_codes_binary, axis=1)

    # Count how often each component is active
    active_per_component = np.sum(sparse_codes_binary, axis=0)

    # Statistics
    n_samples, hidden_size = sparse_codes.shape

    print(f"\nDataset: {n_samples} samples, {hidden_size} sparse dimensions")
    print(f"Tolerance: {tolerance}")

    print(f"\nSparsity per Sample:")
    print(f"  Mean active components: {active_per_sample.mean():.2f} / {hidden_size}")
    print(f"  Median active: {np.median(active_per_sample):.0f}")
    print(f"  Min active: {active_per_sample.min()}")
    print(f"  Max active: {active_per_sample.max()}")
    print(f"  Std: {active_per_sample.std():.2f}")
    print(f"  Sparsity: {(1 - active_per_sample.mean()/hidden_size)*100:.1f}%")
```

```
        print(f"\nComponent Usage:")
        print(f"  Components never used: {np.sum(active_per_component == 0)}")
        print(f"  Components always used: {np.sum(active_per_component == n_samples)}")
        print(f"  Mean usage: {active_per_component.mean():.1f} samples ({active_per_component.mean()/n_samples*100:.1f}%)")

        # Histogram of sparsity
        plt.figure(figsize=(12, 4))

        plt.subplot(1, 3, 1)
        plt.hist(active_per_sample, bins=30, edgecolor='black', alpha=0.7)
        plt.axvline(active_per_sample.mean(), color='red', linestyle='--', label='Mean')
        plt.xlabel('Number of Active Components')
        plt.ylabel('Frequency')
        plt.title('Sparsity Distribution per Sample')
        plt.legend()

        plt.subplot(1, 3, 2)
        plt.hist(active_per_component, bins=30, edgecolor='black', alpha=0.7)
        plt.axvline(active_per_component.mean(), color='red', linestyle='--', label='Mean')
        plt.xlabel('Number of Samples Using Component')
        plt.ylabel('Frequency')
        plt.title('Component Usage Distribution')
        plt.legend()

        plt.subplot(1, 3, 3)
        # Show top 10 most active components
        top_components = np.argsort(active_per_component)[-10:]
        plt.barh(range(10), active_per_component[top_components])
        plt.yticks(range(10), [f'Comp {i}' for i in top_components])
        plt.xlabel('Usage Count')
        plt.title('Top 10 Most Used Components')

        plt.tight_layout()
        plt.savefig('sparsity_analysis.png', dpi=150, bbox_inches='tight')
        print("\n✓ Visualization saved as 'sparsity_analysis.png'")
        plt.show()

        return {
            'active_per_sample': active_per_sample,
            'active_per_component': active_per_component,
            'mean_sparsity': active_per_sample.mean(),
            'sparsity_percentage': (1 - active_per_sample.mean()/hidden_size)*100,
        }
```

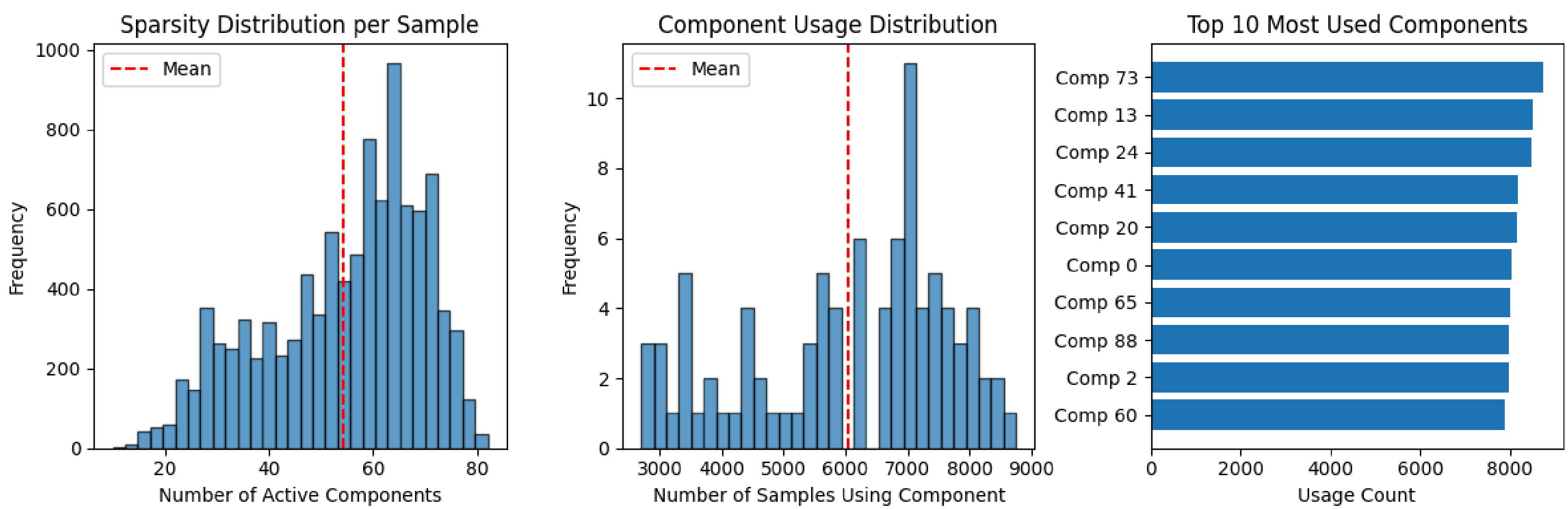
Task 1: Sparsity Analysis

Purpose:

Understand how sparse the learned representations are and which components are most active.

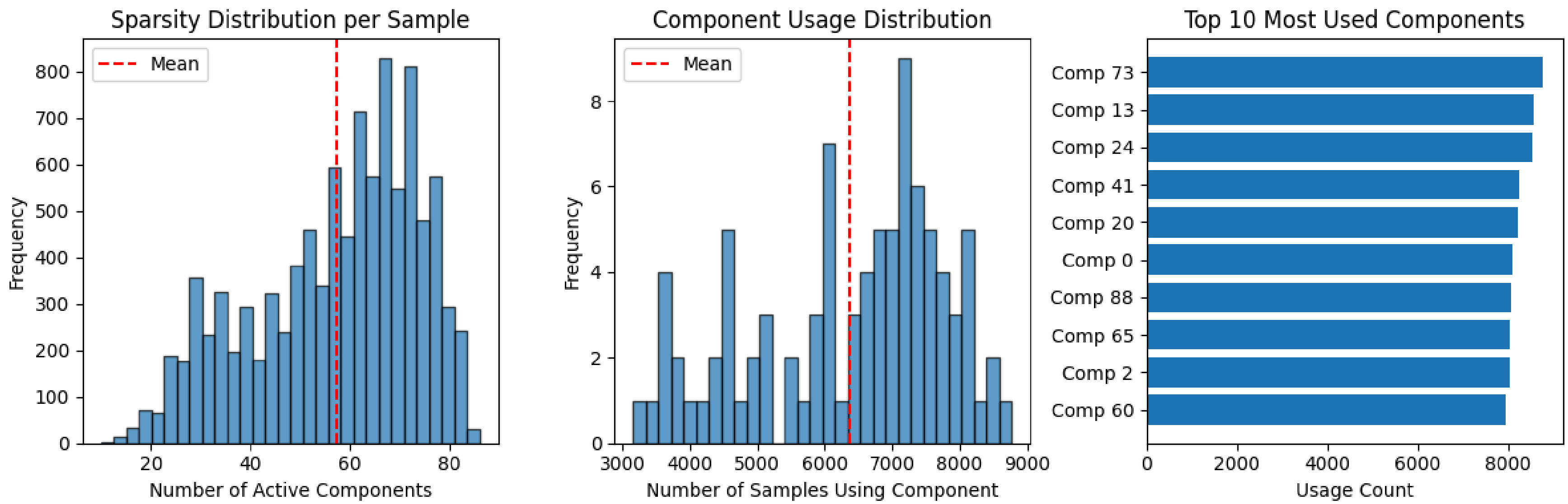
```
# After training KDS model and extracting sparse codes
for model in kds_model_names:
  print(f"Model Name: {model}")
  sparse_codes = model_sparse_codes[model]
  # Analyze sparsity with different tolerances
  for tolerance in [1e-4, 1e-5, 1e-6]:
      sparsity_stats = analyze_sparsity(sparse_codes, tolerance=tolerance)
```

```
Model Name: 0.0001x sparse
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 0.0001

Sparsity per Sample:
  Mean active components: 54.37 / 90
  Median active: 58
  Min active: 10
  Max active: 82
  Std: 15.03
  Sparsity: 39.6%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6041.3 samples (60.4%)

✓ Visualization saved as 'sparsity_analysis.png'
```

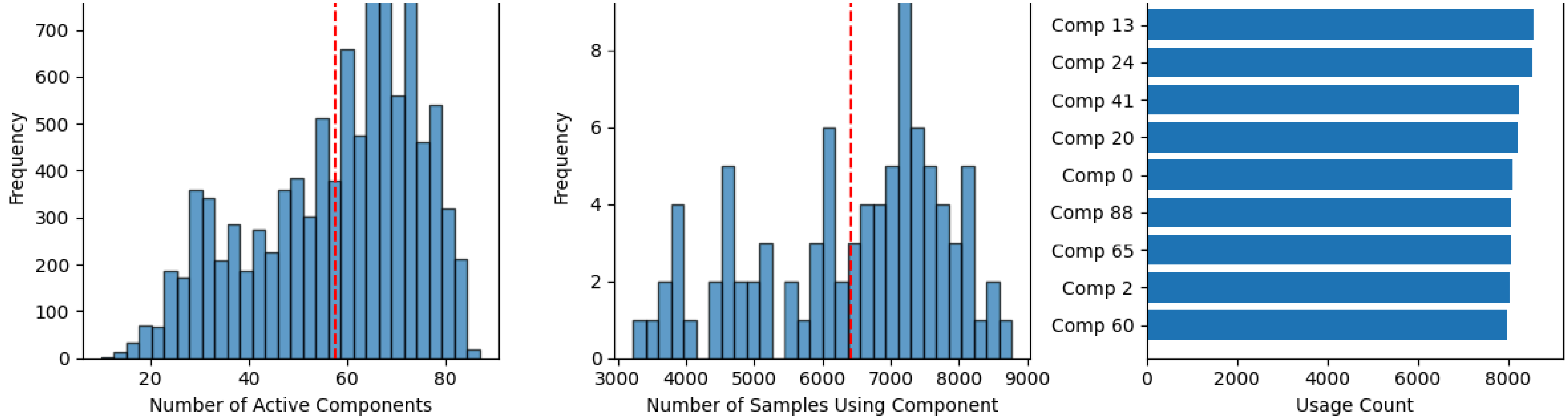



```
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 1e-05

Sparsity per Sample:
  Mean active components: 57.25 / 90
  Median active: 61
  Min active: 10
  Max active: 86
```

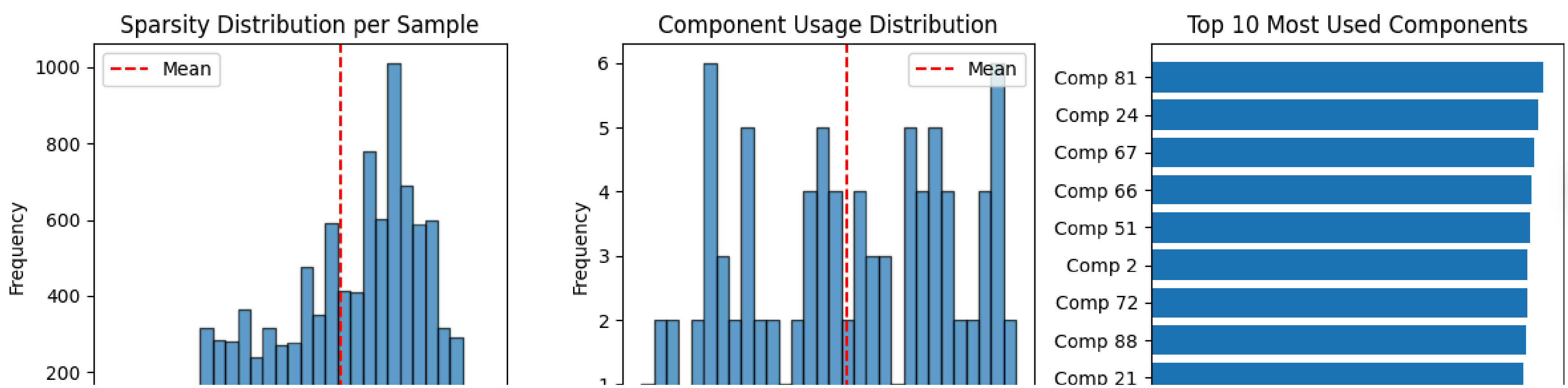
```
  Std: 16.36
  Sparsity: 36.4%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6360.9 samples (63.6%)

✓ Visualization saved as 'sparsity_analysis.png'
```

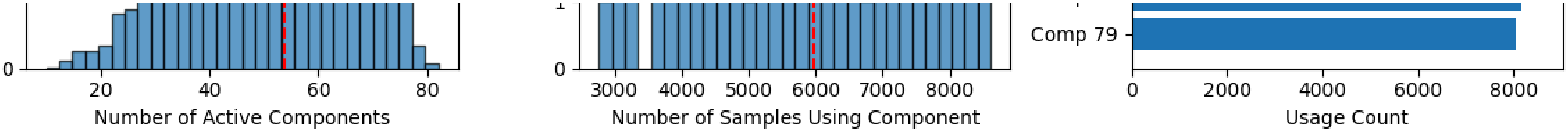



```
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 1e-06

Sparsity per Sample:
  Mean active components: 57.73 / 90
  Median active: 61
  Min active: 10
  Max active: 87
  Std: 16.60
  Sparsity: 35.9%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6414.2 samples (64.1%)

✓ Visualization saved as 'sparsity_analysis.png'
```

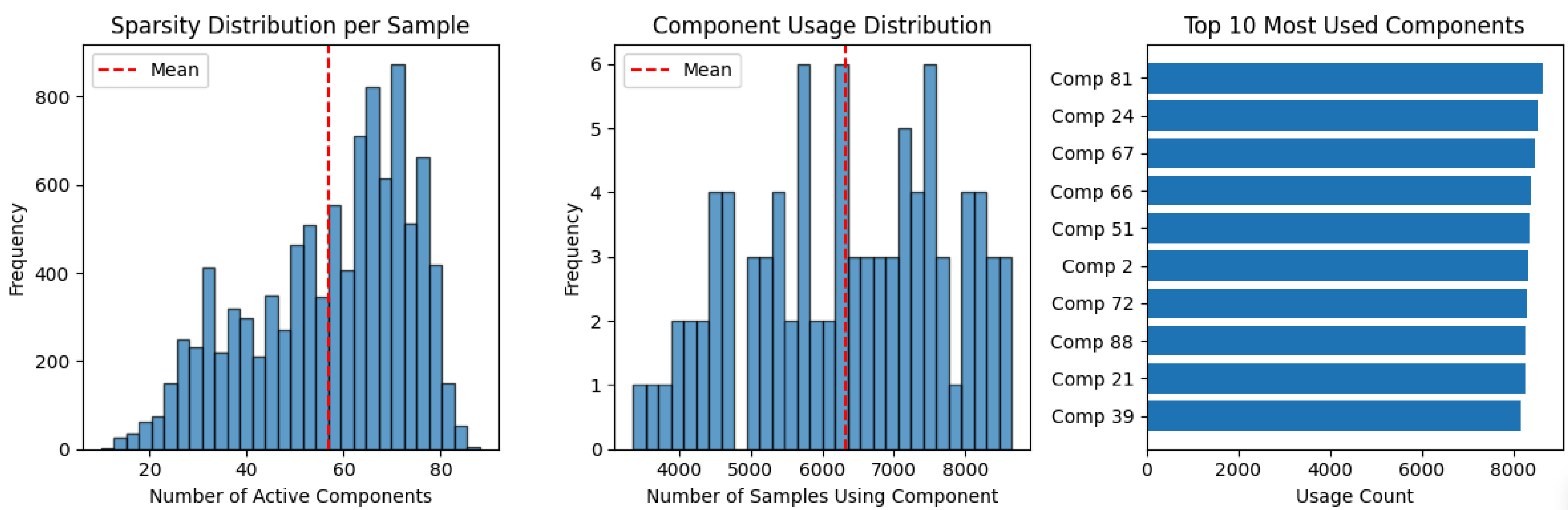



```
Model Name: 0.00001x sparse
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 0.0001

Sparsity per Sample:
  Mean active components: 53.86 / 90
  Median active: 57
  Min active: 10
  Max active: 82
  Std: 14.80
  Sparsity: 40.2%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 5984.6 samples (59.8%)

✓ Visualization saved as 'sparsity_analysis.png'
```



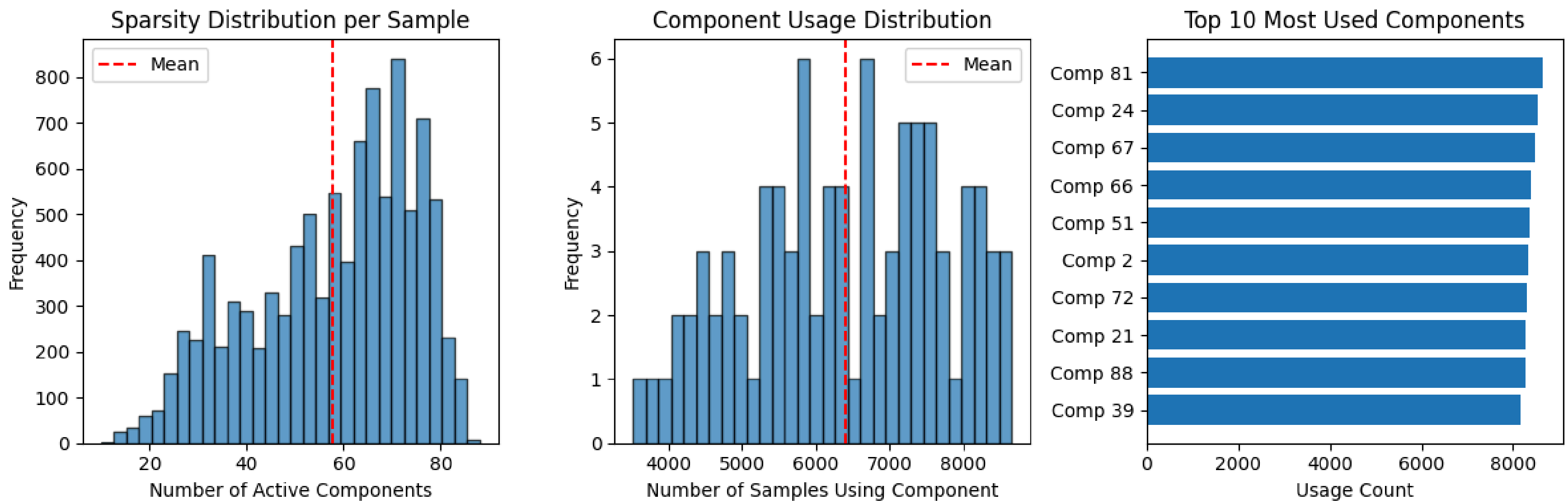



```
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 1e-05

Sparsity per Sample:
  Mean active components: 56.96 / 90
  Median active: 61
  Min active: 10
  Max active: 88
  Std: 16.25
  Sparsity: 36.7%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6329.3 samples (63.3%)

✓ Visualization saved as 'sparsity_analysis.png'
```



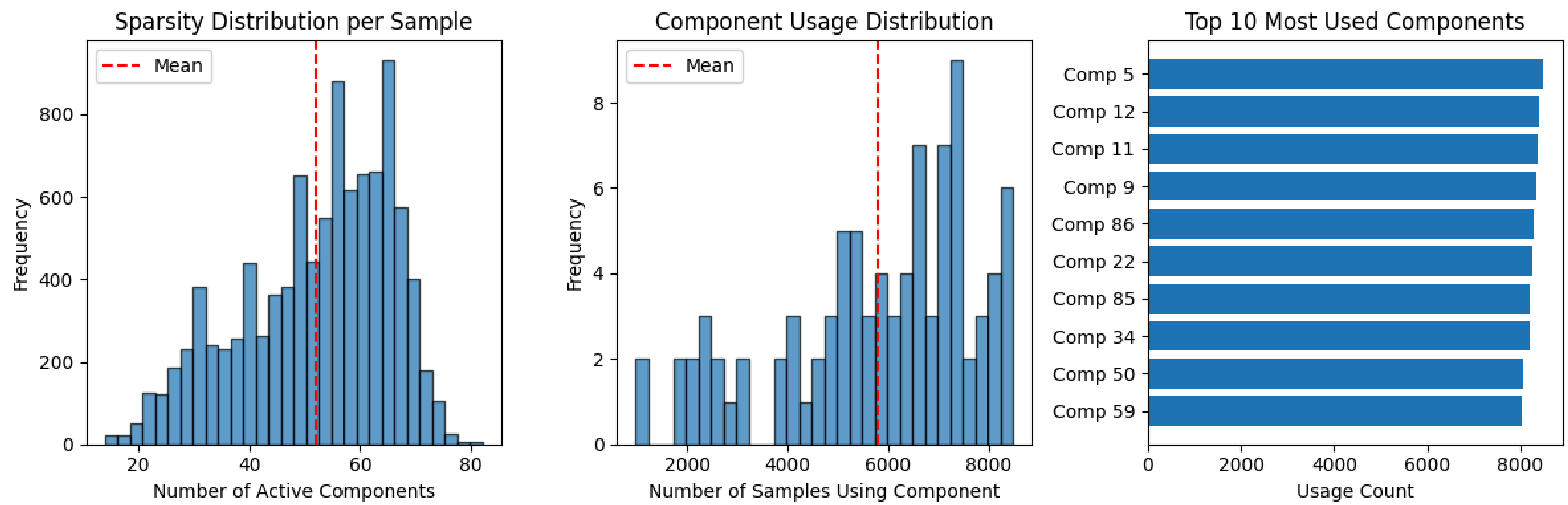

```
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 1e-06

Sparsity per Sample:
```

```
Sparsity per Sample:
  Mean active components: 57.66 / 90
  Median active: 61
  Min active: 10
  Max active: 88
  Std: 16.67
  Sparsity: 35.9%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6406.9 samples (64.1%)

✓ Visualization saved as 'sparsity_analysis.png'
```

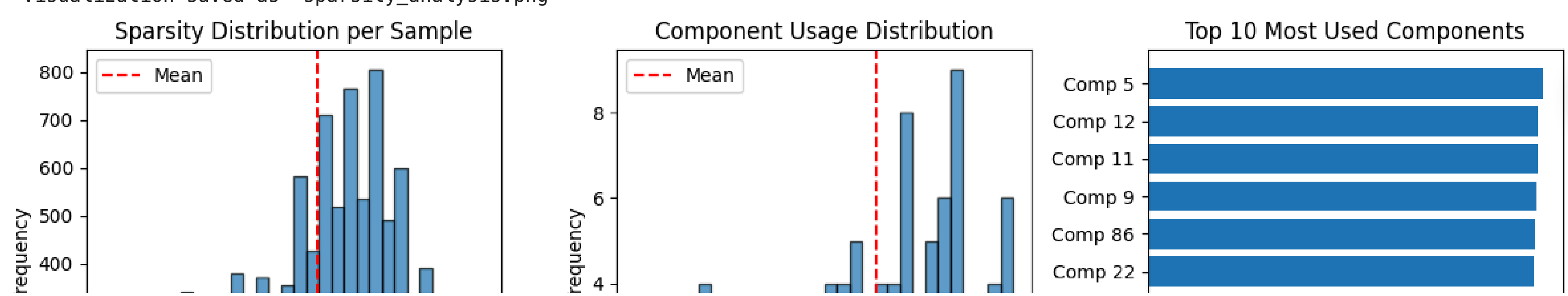


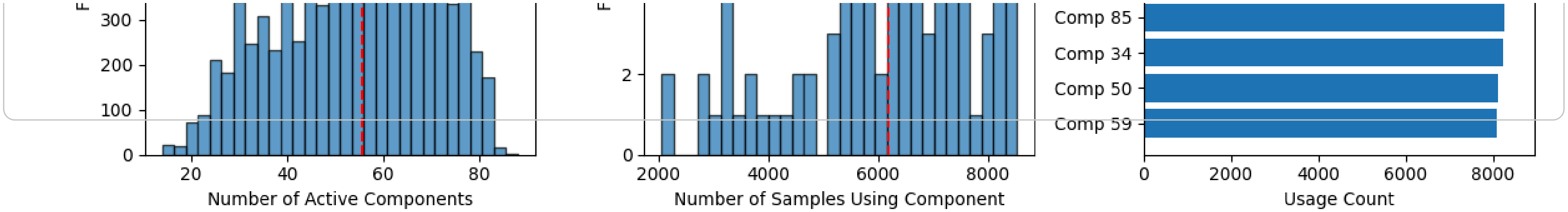

```
Model Name: 0.000005x sparse
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 0.0001

Sparsity per Sample:
  Mean active components: 52.04 / 90
  Median active: 55
  Min active: 14
  Max active: 82
  Std: 13.33
  Sparsity: 42.2%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 5782.4 samples (57.8%)
```

```
✓ Visualization saved as 'sparsity_analysis.png'
```

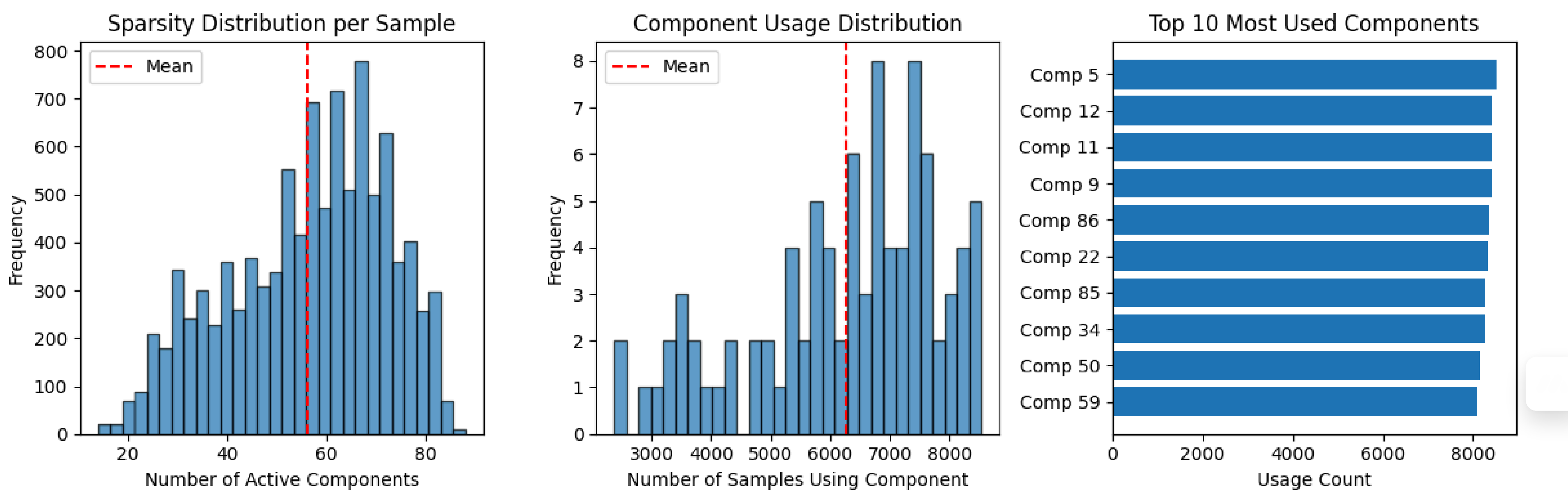



```
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 1e-05

Sparsity per Sample:
  Mean active components: 55.56 / 90
  Median active: 58
  Min active: 14
  Max active: 88
  Std: 15.43
  Sparsity: 38.3%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6173.2 samples (61.7%)

✓ Visualization saved as 'sparsity_analysis.png'
```






```
======================================================================
TASK 1: SPARSITY ANALYSIS
======================================================================

Dataset: 10000 samples, 90 sparse dimensions
Tolerance: 1e-06

Sparsity per Sample:
  Mean active components: 56.27 / 90
  Median active: 59
  Min active: 14
  Max active: 88
  Std: 15.90
  Sparsity: 37.5%

Component Usage:
  Components never used: 0
  Components always used: 0
  Mean usage: 6252.1 samples (62.5%)

✓ Visualization saved as 'sparsity_analysis.png'
```



Take the sparse features into a Ridge Regression model.

```
import numpy as np
from sklearn.linear_model import LinearRegression

epsilon = 1e-6

for model_name in kds_model_names:
  i = kds_model_names.index(model_name)
  net = kds_models[i]
  with torch.no_grad():
    # Extract sparse codes from your trained KDS model
    net.eval()
    sparse_codes_train = net.encode(torch.FloatTensor(X_reduced_train)).detach().numpy()
    sparse_codes_test = net.encode(torch.FloatTensor(X_reduced_test)).detach().numpy()

    # Train linear regression on sparse codes
    lr = LinearRegression()
    lr.fit(sparse_codes_train, y_reduced_train)
    y_pred = lr.predict(sparse_codes_test)

    # Evaluate (log space)
    rmse_log = np.sqrt(mean_squared_error(y_reduced_test, y_pred))
    r2 = r2_score(y_reduced_test, y_pred)

    # Evaluate (dollar space)
    y_test_dollars = np.expm1(y_reduced_test) + epsilon # Division by zero errors.
    y_pred_dollars = np.expm1(y_pred)
    mae_dollars = mean_absolute_error(y_test_dollars, y_pred_dollars)

    y_pred_dollars = price_scaler.inverse_transform(y_pred).ravel()
    rmse = np.sqrt(mean_squared_error(y_reduced_test, y_pred_dollars))
    mape = np.mean(np.abs((y_test_dollars - y_pred_dollars) / y_test_dollars)) * 100 # Mean Absolute Percentage error.

    # Predictive Groups to gauge price representations
    within_5k = np.sum(np.abs(y_test_dollars - y_pred_dollars) < 5000) / len(y_pred) * 100
    within_10k = np.sum(np.abs(y_test_dollars - y_pred_dollars) < 10000) / len(y_pred) * 100
    within_15k = np.sum(np.abs(y_test_dollars - y_pred_dollars) < 15000) / len(y_pred) * 100

    print("="*60)
    print(f"LINEAR REGRESSION ON KDS SPARSE CODES for KDS model {model_name}")
    print("="*60)
    print(f"\n Model Coefficients: ")
    print(f"Current model coefficients for {model_name} - {lr.coef_}")
    print(f"RMSE (log): {rmse_log:.4f} | RMSE: ${rmse:.4f} | R²: {r2:.4f}")
    print(f"MAE: ${mae_dollars:,.0f}")
    print(f"MAPE: {mape:.2f}%")
    print(f"\nPredictive Performance:")
    print(f"  Within ±$5,000:  {within_5k:.1f}%")
```

```
    print(f"  Within ±$10,000: {within_10k:.1f}%")
    print(f"  Within ±$15,000: {within_10k:.1f}%")
    print("="*60)
```

```
============================================================
LINEAR REGRESSION ON KDS SPARSE CODES for KDS model 0.0001x sparse
============================================================

 Model Coefficients:
Current model coefficients for 0.0001x sparse - [[-1.115498   -0.1387223  -2.0531807  -0.09703067  0.93588793 -0.20125568
  -1.5584557  -0.1017682  -0.6299826  -0.42412353 -0.3106112   0.2977442
   0.3391652   0.545172   -0.04973835 -0.58630794  0.02028513  0.07419831
   0.2906441   0.46369517  1.037991   -1.0463107   0.1851806   0.9793134
  -0.24303818 -1.0504898   0.5129325   0.8715913   0.28247866 -0.31520855
  -0.05550098 -1.1363251   1.5864904  -0.54811734  0.5820565   0.5374795
   0.41108975  0.70617646 -0.33053243 -0.23135358 -0.66142106 -0.34698838
  -0.7461164  -0.35185063  0.05508429 -0.4461619  -0.11896634  0.1033968
   0.29914528  0.79470176  0.04617625  0.7197642  -0.90805924  0.48427236
  -0.03224009 -0.35552776  0.32919776 -1.0671902   0.0753274   0.3315326
  -0.9025584  -0.5823989  -0.628601    0.62586135  0.19105631  0.04601144
   1.928695   -0.07654263  0.26370943 -0.4037971   0.44097024 -0.50255847
  -1.0358362   0.6141848  -0.11747928  0.07193974 -0.03909412 -0.99520874
  -0.49232543  0.39301088 -0.04941362  0.8284688   0.98450255  0.47491777
   0.39756563  1.4230057   1.0539542  -0.32193238  1.9127314  -2.1429427 ]]
RMSE (log): 0.2254 | RMSE: $20388.9266 | R²: 0.4090
MAE: $0
MAPE: 112097877235.87%

Predictive Performance:
  Within ±$5,000:  32800.0%
  Within ±$10,000: 190900.0%
  Within ±$15,000: 190900.0%
============================================================
============================================================
LINEAR REGRESSION ON KDS SPARSE CODES for KDS model 0.00001x sparse
============================================================

 Model Coefficients:
Current model coefficients for 0.00001x sparse - [[-0.6648959  -0.11160336  1.9382808   0.11614269  1.1247227  -0.35119593
   0.14616743 -0.44787586  0.675023   -0.75619787  0.18953624  0.02044499
  -0.03909644 -0.45648026 -0.6909107  -1.008223   -0.32254434 -0.95914775
   0.17959577  0.3221184   1.340328   -1.6253979  -0.266482    1.2985008
  -0.1249311  -0.4146592   0.15617475  0.42281544 -0.40745574  0.12243161
   0.43888968  0.6189399   0.01680878  0.08120859 -0.7245712   1.010657
  -0.5651102  -0.23049378  0.79514086  1.1507446  -0.17128766  0.5059652
   0.2574996  -0.18240201  0.17951259 -0.5374097   0.20470388 -1.7305021
   1.2253625   1.319407   -0.19067438 -1.3461634  -0.17988518 -0.94904673
   0.70388824 -0.37566602 -0.26700503 -0.29636577  0.4357593   0.16512065
   0.5675766   0.40128258  0.42118424 -0.442742    0.6020988  -0.28742382
   0.6799429   1.7147367  -0.68308413  0.17536408 -1.0511523  -0.41639346
  -1.5203136  -0.0415947   1.189595    0.7017132  -0.7649759  -0.9048
   0.2477407   0.05510402 -0.98751485 -1.4764556   0.92070615 -0.05779201
  -0.4382732   0.27684546  0.15450305 -1.2122644   0.5885106   0.81967056]]
RMSE (log): 0.2254 | RMSE: $28295.5160 | R²: 0.4090
```

```
MAE: $0
MAPE: 116831124612.68%

Predictive Performance:
  Within ±$5,000:  32600.0%
  Within ±$10,000: 193000.0%
  Within ±$15,000: 193000.0%
============================================================
```

Sparse Codes Training.

```
# Extract sparse codes for FULL 253K training set
print("Extracting sparse codes for full dataset...")
batch_size = 1000

for model_name in kds_model_names:
  i = kds_model_names.index(model_name)
  net = kds_models[i]
  # Train set
  print(f"\nExtracting train sparse codes ({len(X_train):,} samples)...")
  sparse_codes_full_data = []
  with torch.no_grad():
      for i in tqdm(range(0, len(X_train), batch_size)):
          batch = X_train[i:i+batch_size]
          batch_tensor = torch.FloatTensor(batch).to(device)
          codes = net.encode(batch_tensor).cpu().numpy()
          sparse_codes_full_data.append(codes)

  sparse_codes_train = np.vstack(sparse_codes_full_data)
  print(f"Shape: {sparse_codes_train.shape}")  # Should be (253993, 101)

  # Test set
  print(f"\nExtracting test sparse codes ({len(X_test):,} samples)...")
  sparse_codes_test_list = []

  with torch.no_grad():
      for i in tqdm(range(0, len(X_test), batch_size), desc="Test batches"):
          batch = X_test[i:i+batch_size]
          batch_tensor = torch.FloatTensor(batch).to(device)
          codes = net.encode(batch_tensor).cpu().numpy()
          sparse_codes_test_list.append(codes)

  sparse_codes_test = np.vstack(sparse_codes_test_list)
  print(f"✓ Test codes: {sparse_codes_test.shape}")

  # Train linear regression on sparse codes
  lr = LinearRegression()
  lr.fit(sparse_codes_train, y_train)
  y_pred = lr.predict(sparse_codes_test)
```

```
  # Evaluate (log space)
  rmse_log = np.sqrt(mean_squared_error(y_test, y_pred))
  r2 = r2_score(y_test, y_pred)

  # Evaluate (dollar space)
  y_test_dollars = np.expm1(y_test) + epsilon # Division by zero errors.
  y_pred_dollars = np.expm1(y_pred)
  mae_dollars = mean_absolute_error(y_test_dollars, y_pred_dollars)


  y_pred_dollars = price_scaler.inverse_transform(y_pred).ravel()
  rmse = np.sqrt(mean_squared_error(y_test, y_pred))
  mape = np.mean(np.abs((y_test_dollars - y_pred_dollars) / y_test_dollars)) * 100 # Mean Absolute Percentage error.

  # Predictive Groups to gauge price representations
  within_5k = np.sum(np.abs(y_test_dollars - y_pred_dollars) < 5000) / len(y_pred) * 100
  within_10k = np.sum(np.abs(y_test_dollars - y_pred_dollars) < 10000) / len(y_pred) * 100
  within_15k = np.sum(np.abs(y_test_dollars - y_pred_dollars) < 15000) / len(y_pred) * 100

  print("="*60)
  print(f"LINEAR REGRESSION ON KDS SPARSE CODES for KDS model {model_name}")
  print("="*60)
  print(f"\n Model Coefficients: ")
  print(f"Current model coefficients for {model_name} - {lr.coef_}")
  print(f"RMSE (log): {rmse_log:.4f} | R²: {r2:.4f}")
  print(f"MAE: ${mae_dollars:,.0f} | RMSE: ${rmse:,.0f}")
  print(f"MAPE: {mape:.2f}%")
  print(f"\nPredictive Performance:")
  print(f"  Within ±$5,000:  {within_5k:.1f}%")
  print(f"  Within ±$10,000: {within_10k:.1f}%")
  print(f"  Within ±$15,000: {within_10k:.1f}%")
  print("="*60)
```

```
Extracting sparse codes for full dataset...

Extracting train sparse codes (253,993 samples)...
100%|██████████| 254/254 [00:31<00:00,  8.19it/s]
Shape: (253993, 90)

Extracting test sparse codes (128,064 samples)...
Test batches: 100%|██████████| 129/129 [00:18<00:00,  6.99it/s]
✓ Test codes: (128064, 90)
```

```
from sklearn.linear_model import Ridge

# Extract sparse codes for FULL 253K training set
print("Extracting sparse codes for full dataset...")
batch_size = 2000

for model_name in kds_model_names:
  i = kds_model_names.index(model_name)
```

```
    net = kds_models[i]

    with torch.no_grad():
      sparse_codes_full_ridge = []
      for i in tqdm(range(0, len(X_train), batch_size)):
          batch = X_train[i:i+batch_size]
          batch_tensor = torch.FloatTensor(batch).to(device)
          codes = net.encode(batch_tensor).cpu().numpy()
          sparse_codes_full_ridge.append(codes)

      sparse_codes_train_ridge = np.vstack(sparse_codes_full_ridge)
      print(f"Shape: {sparse_codes_train_ridge.shape}")

      # Train Ridge on ALL data
      ridge_full = Ridge(alpha=1.0)
      ridge_full.fit(sparse_codes_train_ridge, y_train.ravel())

      # Evaluate
      sparse_codes_test_ridge = net.encode(torch.FloatTensor(X_test).to(device)).cpu().numpy()
      y_pred = ridge_full.predict(sparse_codes_test_ridge)
      r2_new = r2_score(y_test.ravel(), y_pred)

      print(f"\nOld R² (10K samples): 0.42")
      print(f"New R² (253K samples): {r2_new:.4f}")
```

```
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

models = {
    'Ridge': Ridge(alpha=1.0),
    'RandomForest': RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42),
    'GradientBoosting': GradientBoostingRegressor(n_estimators=100, max_depth=5, random_state=42),
}

best_r2 = 0
for model_name in kds_model_names:
  i = kds_model_names.index(model_name)
  net = kds_models[i]
  with torch.no_grad():
    # Extract sparse codes from your trained KDS model
    net.eval()
    sparse_codes_train = net.encode(torch.FloatTensor(X_reduced_train)).detach().numpy()
    sparse_codes_test = net.encode(torch.FloatTensor(X_reduced_test)).detach().numpy()

    for name, model in models.items():
        model.fit(sparse_codes_train, y_reduced_train)
        y_pred = model.predict(sparse_codes_test)
        r2 = r2_score(y_reduced_test, y_pred)
        print(f"KDS Model: {model_name} - Regression Model {name}: R² = {r2:.4f}")
        if r2 > best_r2:
```

```
            best_r2 = r2
            best_model = model
```

```
# Test Gradient Boosting vs Ridge
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import r2_score
import numpy as np

print("Testing Gradient Boosting...")

for model_name in kds_model_names:
  i = kds_model_names.index(model_name)
  net = kds_models[i]
  with torch.no_grad():
    # Extract sparse codes from your trained KDS model
    net.eval()
    sparse_codes_train = net.encode(torch.FloatTensor(X_reduced_train)).detach().numpy()
    sparse_codes_test = net.encode(torch.FloatTensor(X_reduced_test)).detach().numpy()

    # Use Histogram Gradient Boosting (fast on large datasets)
    gb = HistGradientBoostingRegressor(
        max_iter=300,
        max_depth=10,
        learning_rate=0.05,
        random_state=42,
        verbose=1
    )

    # Train on full sparse codes
    gb.fit(sparse_codes_train, y_reduced_train.ravel())

    # Predict
    y_reduced_test_gb = gb.predict(sparse_codes_test)
    r2_gb = r2_score(y_reduced_test.ravel(), y_reduced_test_gb)

    # Convert to dollars
    y_pred_dollars = price_scaler.inverse_transform(y_reduced_test_gb.reshape(-1, 1)).ravel()
    rmse_gb = np.sqrt(mean_squared_error(y_reduced_test, y_pred_dollars))

    print(f"\n{'='*70}")
    print(f"RESULTS:")
    print(f"  Ridge R²: 0.43-0.45")
    print(f"  GB R²:    {r2_gb:.4f}")
    print(f"  Improvement: {r2_gb - 0.45:+.4f}")
    print(f"  RMSE: ${rmse_gb:,.2f}")
    print(f"{'='*70}")

    if r2_gb >= 0.50:
        print("\n✓✓ SUCCESS! Ready for clustering!")
```

```
    else:
        print("\n→ Need further optimization")
```

```
# Test Gradient Boosting vs Ridge
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import r2_score
import numpy as np

print("Testing Gradient Boosting...")

for model_name in kds_model_names:
  i = kds_model_names.index(model_name)
  net = kds_models[i]
  with torch.no_grad():
    # Extract sparse codes from your trained KDS model
    net.eval()
    sparse_codes_train = net.encode(torch.FloatTensor(X_train)).detach().numpy()
    sparse_codes_test = net.encode(torch.FloatTensor(X_test)).detach().numpy()

    # Use Histogram Gradient Boosting (fast on large datasets)
    gb = HistGradientBoostingRegressor(
        max_iter=300,
        max_depth=10,
        learning_rate=0.05,
        random_state=42,
        verbose=1
    )

    # Train on full sparse codes
    gb.fit(sparse_codes_train, y_train.ravel())

    # Predict
    y_test_gb = gb.predict(sparse_codes_test)
    r2_gb = r2_score(y_test.ravel(), y_test_gb)

    # Convert to dollars
    y_pred_dollars = price_scaler.inverse_transform(y_test_gb.reshape(-1, 1)).ravel()
    rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_dollars))

    print(f"\n{'='*70}")
    print(f"RESULTS:")
    print(f"  Ridge R²: 0.43-0.45")
    print(f"  GB R²:    {r2_gb:.4f}")
    print(f"  Improvement: {r2_gb - 0.45:+.4f}")
    print(f"  RMSE: ${rmse_gb:,.2f}")
    print(f"{'='*70}")

    if r2_gb >= 0.50:
        print("\n✓✓ SUCCESS! Ready for clustering!")
```

```
    else:
        print("\n→ Need further optimization")
```

Clustering the Car Data Appropriately

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, SpectralClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy import stats


# ==============================================================================
# ENHANCED: Statistical Significance Testing
# ==============================================================================

def analyze_clusters_enhanced(df, cluster_labels):
    """
    Enhanced cluster analysis with statistical comparisons
    Shows what makes each cluster UNIQUE vs overall population
    """

    df = df.copy()
    df['cluster'] = cluster_labels
    n_clusters = len(np.unique(cluster_labels))

    print("\n" + "="*70)
    print("ENHANCED CLUSTER ANALYSIS")
    print("="*70)

    summaries = []

    # Overall statistics for comparison
    overall_stats = {
        'price_median': df['price'].median() if 'price' in df.columns else None,
        'age_mean': df['age'].mean() if 'age' in df.columns else None,
        'mileage_mean': df['odometer'].mean() if 'odometer' in df.columns else None,
    }

    for c in range(n_clusters):
        cluster_data = df[df['cluster'] == c]
        n = len(cluster_data)

        print(f"\n{'─'*70}")
        print(f"CLUSTER {c}: {n:,} cars ({100*n/len(df):.1f}%)")
        print(f"{'─'*70}")
```

```
        summary = {'cluster': c, 'size': n, 'pct': 100*n/len(df)}

        # ════════════════════════════════════════════════════════════════════
        # PRICE ANALYSIS with Statistical Comparison
        # ════════════════════════════════════════════════════════════════════
        if 'price' in df.columns:
            cluster_median = cluster_data['price'].median()
            overall_median = overall_stats['price_median']
            diff_pct = ((cluster_median - overall_median) / overall_median) * 100

            print(f"\n💰 PRICE:")
            print(f"   Mean:   ${cluster_data['price'].mean():,.0f}")
            print(f"   Median: ${cluster_median:,.0f} ", end="")

            if abs(diff_pct) > 10:
                direction = "↑" if diff_pct > 0 else "↓"
                print(f"{direction} {abs(diff_pct):.0f}% vs overall")
            else:
                print("(≈ overall average)")

            print(f"   Range:  ${cluster_data['price'].min():,.0f} - ${cluster_data['price'].max():,.0f}")
            print(f"   Std:    ${cluster_data['price'].std():,.0f}")

            summary['median_price'] = cluster_median
            summary['price_vs_overall'] = diff_pct

        # ════════════════════════════════════════════════════════════════════
        # DOMINANT CHARACTERISTICS (What makes this cluster unique?)
        # ════════════════════════════════════════════════════════════════════
        print(f"\n🎯 DOMINANT CHARACTERISTICS:")

        dominant_features = []

        # Check manufacturer concentration
        if 'manufacturer' in df.columns:
            top_mfr = cluster_data['manufacturer'].value_counts()
            top_mfr_name = top_mfr.index[0]
            top_mfr_pct = (top_mfr.values[0] / n) * 100
            overall_mfr_pct = (df['manufacturer'].value_counts()[top_mfr_name] / len(df)) * 100

            if top_mfr_pct > overall_mfr_pct * 1.5:  # 50% more than average
                dominant_features.append(f"{top_mfr_name} ({top_mfr_pct:.0f}%)")
                print(f"   ✓ Dominated by {top_mfr_name}: {top_mfr_pct:.0f}% (overall: {overall_mfr_pct:.0f}%)")

        # Check vehicle type concentration
        if 'type' in df.columns:
            top_type = cluster_data['type'].value_counts()
            top_type_name = top_type.index[0]
            top_type_pct = (top_type.values[0] / n) * 100
            overall_type_pct = (df['type'].value_counts()[top_type_name] / len(df)) * 100
```